



UNIVERSIDAD AUTÓNOMA METROPOLITANA
Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería
Licenciatura en Ingeniería en Computación



Modelo inteligente para la predicción de inventario de productos sensibles al tiempo en microempresas

Proyecto Tecnológico
Segunda versión
Trimestre 2025-Primavera

Alumno

Luis Fernando Robles Pérez
2203031441
al2203031441@azc.uam.mx

Asesora

Dra. Silvia Beatriz González Brambila
Profesora Titular
Departamento de Sistemas
sgb@azc.uam.mx

1 de abril de 2026

Declaratoria

En caso de que el Comité de Estudios de la Licenciatura en Computación apruebe la realización de la presente propuesta, otorgamos nuestra autorización para su publicación en la página de la División de Ciencias Básicas e Ingeniería.

Luis Fernando Robles Pérez
Alumno

Dra. Silvia Beatriz González Brambila
Asesora

Agradecimientos

Quiero expresar mis más sinceros agradecimientos a todas aquellas personas que me han acompañado en este viaje, a quienes han confiado en mí y me han brindado su apoyo. Pero a quienes les debo el mayor reconocimiento es a mis padres **Lilia Pérez González** y **Juan Domingo Robles San Gabriel**, por guiarme con amor, paciencia y orgullo a lo largo de toda mi vida, y por darme la motivación que necesitaba cuando quería rendirme. Gracias a mis mentores, logré cruzar una meta más.

Aunque hoy en día ya no me acompañan las mismas personas que hace 5 años, fueron parte de mi crecimiento y de igual forma merecen un sincero agradecimiento por haber formado parte de mi camino.

Quiero agradecer también a la **Dra. Silvia Beatriz González Brambila**, quien compartió su conocimiento e inspiración para que pueda ser un mejor profesionista.

Por último, quiero agradecer a mis compañeros y amigos que conocí durante mi estancia en la universidad, por hacer más amenos los momentos difíciles, y a la UAM por brindarme enseñanzas y conocimientos.

Resumen

Este proyecto enfrenta una problemática sumamente común en micro-negocios, que es la gestión ineficiente de inventarios, donde la falta de predicción en la demanda puede generar pérdidas por sobrestock, caducidad o desabasto. Se propone el desarrollo de un sistema inteligente de gestión de inventarios que integra análisis de datos históricos, modelos de aprendizaje automático y una arquitectura web moderna para apoyar la toma de decisiones.

El sistema implementa un backend desarrollado en FastAPI con conexión a MySQL, donde se almacenan productos, ventas e inventarios. A partir de datos históricos normalizados, se entrenan modelos de predicción de demanda (como Random Forest y Gradient Boosting), incorporando variables como estacionalidad y eventos relevantes. Estas predicciones permiten estimar ventas futuras por producto y optimizar los niveles de stock.

En el lado del frontend, fue desarrollado con React y TypeScript, se ofrece una interfaz clara e intuitiva que permite registrar ventas, visualizar productos, consultar métricas clave (KPIs) y analizar predicciones mediante gráficos dinámicos. La comunicación entre cliente y servidor se realiza mediante una API REST, permitiendo modularidad y escalabilidad del sistema.

El presente trabajo proporciona una herramienta accesible y basada en datos que ayuda a reducir las pérdidas ocasionadas por mermas, mejora la rotación de productos y optimiza la disponibilidad que se tiene en tienda. Con estas herramientas, el sistema es capaz de automatizar procesos administrativos y aportar inteligencia predictiva al negocio para una gestión más eficiente y sostenible del inventario en pequeños comercios.

Índice

1. Planteamiento del problema	1
1.1. Introducción	1
1.2. Justificación	1
1.3. Objetivos	1
1.3.1. Objetivo general	1
1.3.2. Objetivos específicos	1
1.3.3. Objetivos logrados	2
1.4. Antecedentes	3
1.4.1. Neural Network Control Of Perishable Inventory With Fixed Shelf Life Products And Fuzzy Order Refinement Under Time-Varying Uncertain Demand [1].	3
1.4.2. Estrategias Basadas En Inteligencia Artificial Para La Gestión De Inventarios En La Cadena De Suministro [2].	3
1.4.3. Transformación Digital En La Logística Internacional: Estrategias Y Desafíos De La Inteligencia Artificial Para Los Inventarios Y Cadena De Suministro En Las Empresas Exportadoras Colombianas [3].	3
1.4.4. Analysis Of An Inventory Model In Perishable Products Applying Tabu And Montecarlo Simulation Metaheuristic Algorithm Ecuadorian Science Journal [4].	3
1.4.5. Gestión De Inventario De Artículos Perecederos [5].	4
1.4.6. Herramienta De Optimización Inventario En Un E-Commerce De Bienes Perecederos [6].	4
1.5. Estructura de la documentación	5
1.5.1. Reporte	6
1.5.2. Programa	7
1.5.3. App	8
2. Marco Teórico	10
2.1. Clasificación de productos	10
2.1.1. Productos perecederos	10
2.1.2. Productos no perecederos	10
2.1.3. Productos no perecederos	10
2.1.4. Productos no perecederos	11
2.2. Datos en sistemas de inventario	11
2.2.1. Fuentes de datos	11
2.2.2. Preprocesamiento de datos	11
2.2.3. Ingeniería de características	11
2.3. Modelos de Machine Learning para la predicción de demanda	11
2.3.1. Modelos de regresión	11
2.3.2. Random Forest	12
2.3.3. Redes neuronales y evaluación de modelos	12
2.3.4. Evaluación del modelo	13
2.4. Arquitectura de software	14
2.4.1. Modelo MVC	14
2.4.2. Backend con FastAPI	15

2.4.3. API REST	15
2.5. Desarrollo frontend con React	15
3. Desarrollo del proyecto	16
3.1. Obtención de datasets	16
3.2. Preprocesamiento y clasificación de productos	17
3.3. Obtención de datos de eventos y festividades	18
3.4. Integración de temporadas de productos perecederos	18
3.5. Generación de dataset de eventos asociados a productos	19
3.6. Normalización del dataset de ventas	19
3.7. Entrenamiento del modelo predictivo	19
3.8. Diseño de la base de datos	20
3.8.1. Modelo relacional	20
3.8.2. Estructura de tablas principales	21
3.9. Implementación del backend	22
3.9.1. Estructura del sistema	23
3.9.2. Integración del modelo predictivo	23
3.10. Implementación del frontend	23
3.10.1. Arquitectura general del frontend	23
4. Resultados	24
4.1. Evaluación del modelo predictivo	24
4.2. Justificación del desempeño del modelo	25
4.3. Evaluación del backend	25
4.4. Implementación del frontend	26
4.4.1. Vistas principales del sistema	26
5. Análisis y discusión de resultados	33
5.1. Interpretación de resultados	33
5.2. Desempeño del sistema	33
5.3. Valor agregado del proyecto	33
5.4. Implicaciones prácticas	33
5.5. Limitaciones y líneas futuras	34
6. Conclusiones	35
Apéndice A. Apéndices	38
A.1. Código fuente del clasificador de productos, clasificador_perecederos.py	38
A.2. Código fuente del módulo obtener_feriados_CDMX.py	42
A.3. Código fuente del módulo normalizacion_fruitsvegetables_temporadas.py	43
A.4. Código fuente del módulo generar_eventos_productos.py	46
A.5. Código fuente del módulo completar_dataSet_ventas.py	50
A.6. Código fuente del módulo entrenamiento_modelo_demanda.py	53
A.7. Lenguaje de Definición de Datos scriptBD.mysql	59
A.8. Código fuente del backend	61
A.8.1. Módulo models	61
A.8.2. Módulo schemas	63
A.8.3. Módulo routers	67

A.8.4. Modulo services	70
A.9. Integración del modelo predictivo con el backend	86

Índice de figuras

1.1. Esquema de directorios principales.	6
1.2. Esquema del directorio Reporte.	7
1.3. Esquema del directorio Programa.	7
2.1. Representación del funcionamiento del algoritmo Random Forest	12
2.2. Representación de una red neuronal	13
2.3. Interaccion del modelo MVC	15
3.1. Esquema general del sistema Modelo inteligente para la predicción de inventario de productos sensibles al tiempo en microempresas	16
3.2. Modelo relacional de la base de datos	21
4.1. Vista del dashboard principal del sistema.	26
4.2. Vista de gestión y consulta de productos.	27
4.3. Alerta de visualización de un producto.	27
4.4. Edición de un producto.	28
4.5. Vista para el registro de ventas.	29
4.6. Vista de consulta del historial de ventas.	30
4.7. Vista del módulo de predicción de demanda.	31
4.8. Vista de consulta del estado del inventario.	32

Índice de cuadros

1. Comparación cualitativa de los antecedentes con el proyecto propuesto. 4

Índice de Fragmentos de Datos

1.	Ejemplo de la estructura generada por la herramienta de inteligencia artificial. . . .	17
2.	Ejemplo del dataset de ventas en formato CSV.	17
3.	Ejemplo del dataset de eventos proporcionados por la API de Calendarific	18
4.	Ejemplo del dataset de productos perecederos	18
5.	Ejemplo del dataset de productos perecederos normalizado	18
6.	Ejemplo del dataset de productos relacionados con su respectiva fecha	19
7.	Ejemplo del dataset de ventas normalizado	19

1. Planteamiento del problema

1.1. Introducción

En comercios locales, específicamente los dedicados a la venta de productos alimenticios como frutas, verduras y otros con vida útil limitada (producto que puede ser almacenado, exhibido y vendido únicamente durante un periodo determinado), es común que algunos de estos perezcan antes de ser vendidos, lo que genera pérdidas económicas. Ciertos productos presentan variaciones en la demanda según la temporada, lo cual puede resultar en escenarios donde el inventario es insuficiente para satisfacerla. Esta situación afecta principalmente a los productos perecederos, debido a su vida útil limitada ya que necesitan de un manejo más preciso para evitar daños, deterioro o vencimiento.

La venta de muchos productos perecederos se ve influenciada por la estación del año, así como factores externos como festividades y condiciones climáticas. A diferencia de los productos no perecederos, estos deben ser vendidos en un periodo corto para evitar mermas. La falta de previsión en la demanda puede ocasionar pérdidas tanto por desabasto como por sobreabasto, lo que afecta negativamente al negocio.

Por ello es importante desarrollar soluciones que permitan anticipar la demanda de forma más precisa. Como respuesta a esta necesidad, se propone el desarrollo de un sistema de predicción de inventario, el cual utilizará técnicas de inteligencia artificial y ciencia de datos. Este estará enfocado principalmente en productos perecederos y considerará variables externas como estacionalidad, clima y eventos especiales para reducir las pérdidas ocasionadas por la mala gestión del inventario y el *stock*.

1.2. Justificación

El manejo adecuado del inventario es fundamental para el crecimiento de cualquier negocio, especialmente cuando se trata de productos perecederos. Una gestión ineficiente puede derivar en pérdidas económicas considerables. Muchos comercios locales no tienen acceso a herramientas tecnológicas que les permitan anticipar con precisión la demanda futura, lo que genera decisiones basadas en suposiciones más que en datos certeros.

Implementar un sistema de predicción basado en inteligencia artificial representa una solución para mejorar la planificación del inventario, optimizar recursos y reducir pérdidas.

1.3. Objetivos

1.3.1. Objetivo general

Desarrollar un sistema inteligente para la gestión de inventarios de micro empresas, con enfoque principal en productos perecederos, que optimice el control del *stock* mediante técnicas de inteligencia artificial y ciencia de datos.

1.3.2. Objetivos específicos

1. Recolectar y limpiar los datos necesarios para el modelo, aplicando técnicas de análisis exploratorio (EDA) y feature engineering (ingeniería de características), para poder construir un conjunto de datos (dataset) optimo que permita entrenar al modelo de forma precisa y eficiente además de asegurar resultados confiables..

2. Clasificar los productos en perecederos y no perecederos para facilitar el análisis y procesamiento de los datos.
3. Determinar características clave de los productos perecederos, como su vida útil, estacionalidad, y otras propiedades relevantes para la predicción de la demanda.
4. Integrar variables externas al modelo que influyen en el comportamiento de compras, como estacionalidad, clima, festividades que afectan la demanda.
5. Desarrollar un modelo predictivo basado en técnicas de aprendizaje automático que permita predecir de manera precisa la futura demanda de cada producto, considerando su comportamiento de ventas históricas y condiciones externas.

1.3.3. Objetivos logrados

Durante el desarrollo del sistema se cumplieron los objetivos planteados en la propuesta inicial, con adaptaciones que favorecieron la eficiencia del procesamiento y la experiencia del usuario final. A continuación, se detallan los principales logros alcanzados:

- Se realizó la recolección, integración y limpieza de múltiples datos relacionadas con productos, ventas y otra información relevante, aplicando técnicas de análisis exploratorio de datos. Esto permitió construir un conjunto de datos estructurado y fiable para el entrenamiento de distintos modelos predictivos.
- Se implementó un proceso de clasificación de productos que permite distinguir entre productos perecederos y no perecederos, por lo que facilita su análisis y permite aplicar estrategias específicas para la gestión del inventario para cada tipo de producto.
- Se identificaron y modelaron características relevantes de los productos perecederos, tales como vida útil, temporalidad de consumo y patrones de venta, lo que permite entender mejor cómo se venden los productos y cómo se gestionan dentro del inventario.
- Se incorporaron factores externos que influyen en la demanda de productos, como estacionalidad, eventos y festividades, con el objetivo de mejorar la capacidad predictiva del sistema.
- Se desarrollaron y evaluaron modelos de aprendizaje automático para la predicción de la demanda, utilizando datos históricos de ventas y variables contextuales. Estos modelos permiten estimar las ventas futuras por producto, lo que ayuda a mejorar la toma de decisiones en la gestión del inventario.
- Se diseñó e implementó una arquitectura de software basada en un backend desarrollado con *FastAPI* y una base de datos relacional implementada en *MySQL*, encargada de la gestión de productos, inventarios y ventas.
- Se desarrolló una interfaz web interactiva utilizando *React* y *TypeScript*, que permite al usuario registrar ventas, administrar productos, consultar inventarios y visualizar información relevante para la toma de decisiones (predicciones).
- Se integró el sistema completo en una Aplicación Web que combina gestión de inventario, análisis de datos y predicción de demanda, proporcionando una herramienta útil para microempresas que manejan productos perecederos.

1.4. Antecedentes

1.4.1. Neural Network Control Of Perishable Inventory With Fixed Shelf Life Products And Fuzzy Order Refinement Under Time-Varying Uncertain Demand [1].

El estudio propone un Sistema de Control Fuzzy Robust Neural Network (FRNN) para optimizar inventarios de productos perecederos con demanda incierta, con el uso de Redes Neuronales Robustas (RNN) y Lógica Difusa. Los resultados muestran reducciones de desperdicio (hasta 74 % en demanda impredecible), mayores tasas de cumplimiento (hasta 14 % en demanda irregular), menores niveles de *stock* (24 % menos) y mejor desempeño general (83 % de los casos) frente a métodos tradicionales, mejorando rentabilidad y sostenibilidad según datos de un minorista global.

1.4.2. Estrategias Basadas En Inteligencia Artificial Para La Gestión De Inventarios En La Cadena De Suministro [2].

El estudio propone el uso de inteligencia artificial (aprendizaje automático y algoritmos genéticos) para optimizar la gestión de inventarios en cadenas de suministro, mejorando la predicción de demanda, reduciendo costos, incluyendo transporte y desperdicios, y aumentando la disponibilidad de productos. Los resultados muestran mayor precisión en pronósticos, optimización de inventarios, eficiencia operativa y satisfacción del cliente, demostrando el gran potencial que tiene la IA en este ámbito.

1.4.3. Transformación Digital En La Logística Internacional: Estrategias Y Desafíos De La Inteligencia Artificial Para Los Inventarios Y Cadena De Suministro En Las Empresas Exportadoras Colombianas [3].

Este estudio abordó los problemas de gestión de inventarios en empresas exportadoras colombianas, como pronósticos inexactos de demanda, desequilibrios de *stock*, altos costos logísticos y baja competitividad global, con el objetivo de mejorar su eficiencia operativa, reducir costos (almacenamiento, transporte), minimizar errores y fortalecer su posición en el mercado internacional. Para solucionarlo, se implementó Inteligencia Artificial mediante algoritmos de aprendizaje automático, automatización de procesos y optimización de rutas en tiempo real (considerando tráfico y clima). Los resultados incluyeron mayor precisión en pronósticos, reducción de costos operativos y de transporte, mejor optimización de inventarios y una mejor adaptación a los cambios constantes del mercado, lo que incrementó la eficiencia y competitividad de estas empresas en el ámbito global.

1.4.4. Analysis Of An Inventory Model In Perishable Products Applying Tabu And Montecarlo Simulation Metaheuristic Algorithm Ecuadorian Science Journal [4].

El estudio resolvió el problema de gestión ineficiente de inventarios de roscas en la panadería “El Chino”, donde el desconocimiento de cantidades y frecuencias de producción generaba exceso de *stock*, ventas a precio de costo de producción y pérdidas por caducidad. Para darle solución a este problema, se aplicaron los algoritmos Tabú y simulación Montecarlo con datos históricos de ventas (2017-2019), usando herramientas accesibles como Excel y análisis de distribución Log Normal. Los resultados mostraron una reducción drástica del inventario final (485 unidades con Montecarlo y 53 adicionales con Tabú), demostrando su eficacia para mejorar la gestión de inventarios y proponiendo estas técnicas como la mejor opción para su optimización integral.

1.4.5. Gestión De Inventario De Artículos Perecederos [5].

El estudio resolvió el problema de gestión de inventarios para productos perecederos con demanda dinámica, capacidad limitada y costos fijos. Al comprobar que las redes de flujo no eran viables para modelar el problema, se desarrolló un enfoque aproximado que combinó un algoritmo de Hsu modificado (para soluciones iniciales factibles) con tres heurísticas de mejora (Shift de Producción, Parcial y de Saturación), implementadas en Python con NumPy. Los resultados demostraron que esta solución heurística es eficiente y escalable, obteniendo en muchos casos resultados cercanos o incluso superiores a los métodos exactos (como GLPK) en tiempos computacionalmente viables, especialmente cuando se aplicaban las tres heurísticas conjuntamente, ofreciendo así una alternativa práctica para problemas de gran escala.

1.4.6. Herramienta De Optimización Inventario En Un E-Commerce De Bienes Perecederos [6].

El estudio resolvió el problema de gestión ineficiente de inventario de flores en un *E-Commerce* vietnamita, donde la falta de herramientas precisas causaba pérdidas del 15-20 % por desperdicios, con el objetivo de optimizar compras diarias, reducir costos y mejorar la rentabilidad. La solución consistió en desarrollar una herramienta en Google Sheets con cinco módulos interconectados: predicción de *stock* óptimo, gestión de productos/proveedores, seguimiento de precios y costos. Los resultados mostraron: reducción esperada del 20 % en desperdicios, mejor seguimiento de inventario, análisis de tendencias de demanda, ahorro de tiempo laboral y una solución escalable para futuras expansiones.

Las similitudes y diferencias con el proyecto propuesto, se presentan en la Tabla 1.

Tabla 1. Comparación cualitativa de los antecedentes con el proyecto propuesto.

Ref.	Similitudes	Diferencias
[1]	■ Busca minimizar las pérdidas económicas y el desperdicio de productos ■ Pretende mejorar la eficacia operativa y la rentabilidad ■ Ambos consideran patrones de demanda	■ Este proyecto se enfoca en comercios locales con productos como frutas y verduras mientras que [1] aborda un inventario a nivel de cadena de suministro global ■ A diferencia del proyecto [1], que considera como variable el deterioro del producto durante el transporte, este proyecto no contempla variables como el tiempo de traslado ni ningún aspecto logístico.
[2]	■ Ambos proyectos buscan mejorar la gestión del <i>stock</i> en comercios locales o en empresas exportadoras ■ Se busca aplicar técnicas de inteligencia artificial para prever la demanda con mayor precisión y apoyar a la toma de decisiones	■ El proyecto [2] se enfoca en inventarios y cadenas de suministro en “empresas exportadoras colombianas” lo que implica una complejidad de logística internacional ■ Este proyecto se enfoca exclusivamente en la gestión del inventario, mientras el proyecto [2] abarca más tareas como la automatización de tareas administrativas, mejorar la planificación de rutas, entre otras más.

Continuación de la Tabla 1

Ref.	Similitudes	Diferencias
[3]	■ Ambos proyectos identifican patrones en datos históricos y en tiempo real para prever la demanda. ■ Los dos proyectos buscan gestionar el inventario para ahorrar costos por merma o exceso de <i>stock</i>.	■ El proyecto [3] está orientado a la logística de exportación global en empresas colombianas ■ El proyecto [3] contempla actividades mas complejas como la automatización de tareas como el procesamiento inteligente de documentos y gestión de riesgos logísticos
[4]	■ Ambos proyectos se enfocan en los dos tipos de productos, perecederos y no perecederos. ■ Ambos modelos se preocupan por reducir las pérdidas causadas por la merma.	■ El trabajo [4] utiliza herramientas accesibles como Excel y algoritmos simples. ■ Este proyecto esta orientado a comercios locales, mientras que el trabajo [4] se centra en comercios muy concretos como una panadería o supermercado específico
[5]	■ Ambos proyectos coinciden en reducir pérdidas económicas y el desperdicio por mala gestión del inventario.	■ El trabajo [5] analiza una panadería real y un producto específico (roschas) mientras que esta propuesta plantea una solución general a comercios locales
[6]	■ Ambos sistemas buscan predecir las necesidades futuras basándose en históricos de ventas y demanda. ■ Ambos estudios su principal objetivo es gestionar el inventario de productos con vida útil limitada como los alimentos o flores	■ El proyecto [6] utiliza herramientas como Google Sheets con fórmulas y sin utilizar IA compleja. ■ Este proyecto propone tomar decisiones automáticas con modelos predictivos, mientras que el proyecto [6] requiere intervención humana para ajustar parámetros.

1.5. Estructura de la documentación

El directorio principal denominado *ProyectoIntegrador_RoblesPerezLuisFernando_2203031441* contiene todos los entregables del proyecto y esta organizado en tres directorios principales

- App
- Programa
- Reporte

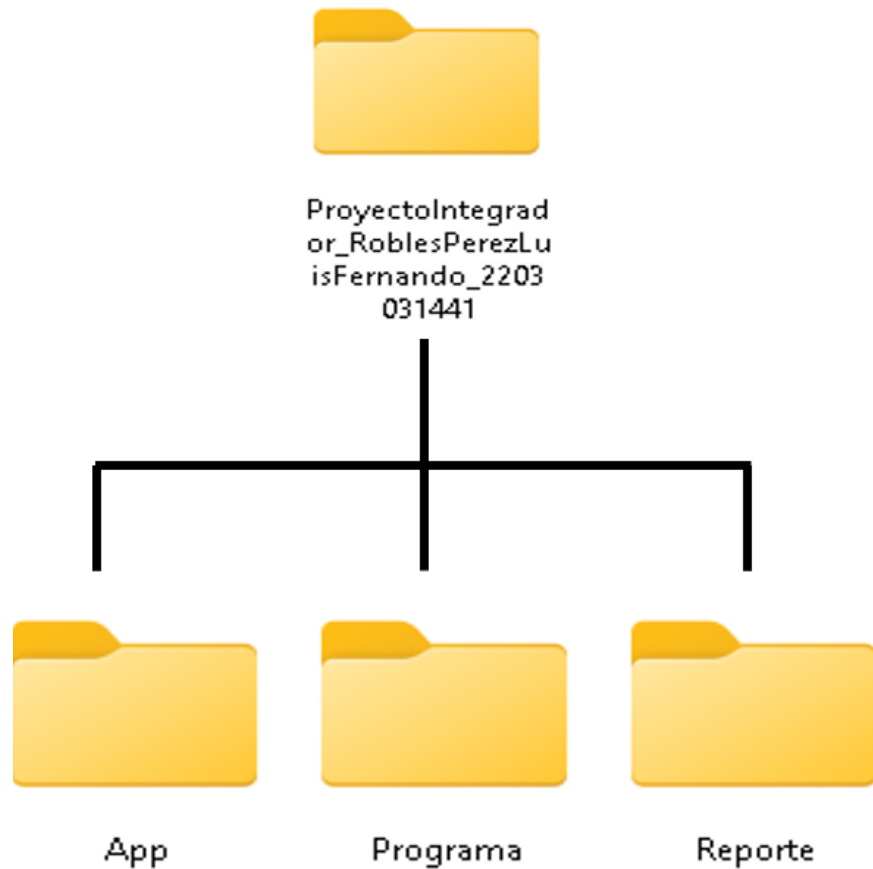


Figura 1.1. Esquema de directorios principales.

1.5.1. Reporte

En este directorio se almacena el documento del reporte final en formato PDF, sin restricciones de acceso. Su estructura interna es la siguiente:

1. Portada
2. Agradecimientos
3. Índice
4. Tablas de contenido (tablas, figuras y ecuaciones)
5. Introducción (justificación, objetivos, antecedentes y estructura del documento)
6. Marco teórico
7. Desarrollo del proyecto
8. Resultados
9. Análisis y discusión de resultados
10. Conclusiones

11. Referencias

12. Apéndices

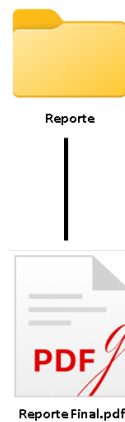


Figura 1.2. Esquema del directorio Reporte.

1.5.2. Programa

El directorio **Programa** almacena el código fuente de la aplicación, así como los archivos de configuración y las variables de entorno necesarias para su ejecución, como se muestra en la figura a continuación.

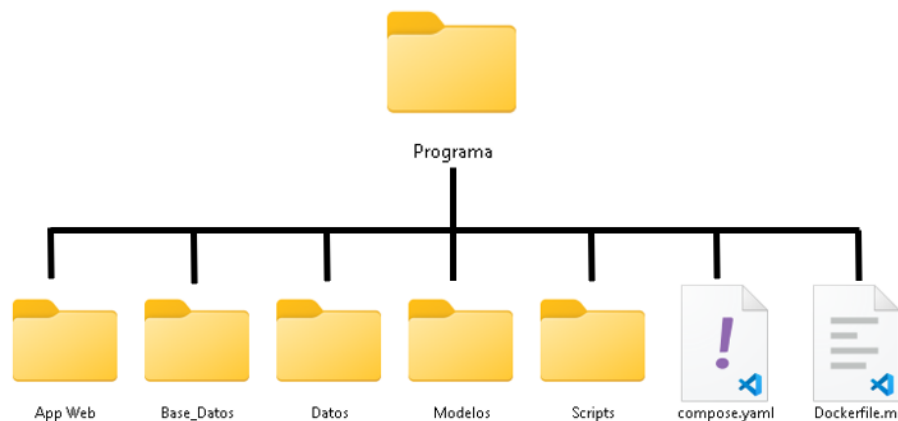


Figura 1.3. Esquema del directorio Programa.

Appweb En este directorio se encuentra el código fuente de la aplicación web. Incluye el *backend*, desarrollado en Python utilizando el framework *FastAPI* y siguiendo una arquitectura de tipo API REST, así como el *frontend* desarrollado con la librería *React* y utilizando *TypeScript*.

Base_Datos Aquí se encuentra un archivo de tipo *MySQL*, el cual contiene el script necesario para crear la base de datos utilizada por el sistema, donde se almacena la información recopilada y procesada.

Datos En este directorio se encuentran tres subdirectorios llamados **datasets_originales**, **datasets_procesados** y **eventos**. En **datasets_originales** se almacenan los conjuntos de datos en su forma original, sin ningún tipo de procesamiento. En **datasets_procesados** se guardan los datasets que ya fueron limpiados, normalizados y preparados para su uso en los modelos. Finalmente, en **eventos** se almacenan los datos relacionados con eventos importantes de la Ciudad de México que pueden influir en el comportamiento de las ventas.

Modelos En este directorio se almacenan los modelos de aprendizaje automático una vez que han sido entrenados. Estos modelos posteriormente son utilizados por el backend de la aplicación web para realizar las predicciones de demanda. Asimismo, aquí se encuentra el script encargado de entrenar el modelo principal utilizado para la predicción de la demanda.

Scripts Este directorio contiene cuatro subdirectorios principales:

- **carga_bd**: Contiene los scripts encargados de cargar la información procesada en la base de datos MySQL.
- **clasificadores**: Incluye los scripts utilizados para generar el modelo que clasifica los productos en perecederos y no perecederos.
- **generacion_datos**: Contiene un script que se conecta a una API de calendario para obtener los días festivos de la Ciudad de México.
- **limpieza_datasets**: Incluye los scripts necesarios para procesar, limpiar y normalizar los datasets que serán utilizados por el modelo de predicción de demanda.

compose.yml Este archivo define y configura los contenedores Docker necesarios para ejecutar los scripts de procesamiento de datos y el entrenamiento de los modelos de inteligencia artificial.

Dockerfile.ml Contiene las instrucciones necesarias para instalar las dependencias del proyecto, las versiones requeridas de Python y configurar el entorno de trabajo dentro del contenedor.

1.5.3. App

En este directorio se encuentra un único archivo llamado *compose.yaml*. Este archivo se encarga de definir y ejecutar los contenedores necesarios para la aplicación web. Crea dos contenedores: uno para ejecutar el backend del sistema y otro encargado de servir el frontend. De esta manera, permite levantar e interactuar con la aplicación web completa de forma sencilla.

A partir de la estructura descrita en las secciones anteriores, el proyecto se organiza en el siguiente árbol de directorios, el cual muestra la distribución general de los componentes del sistema.

Proyecto

- |__ App
- |__ Programa
 - | |__ App Web
 - | | |__ backend
 - | | | |__ app
 - | | | | |__ ml
 - | | | | |__ ml_models
 - | | | | |__ models
 - | | | | |__ routers
 - | | | | |__ schemas
 - | | | | |__ services
 - | | | |__ frontend
 - | | | | |__ public
 - | | | | |__ src
 - | | | | | |__ assets
 - | | | | | |__ components
 - | | | | | | |__ dashboard
 - | | | | | |__ pages
 - | | | | | |__ styles
 - | | |__ Base_Datos
 - | | |__ Datos
 - | | | |__ datasets_originales
 - | | | |__ datasets_procesados
 - | | | |__ eventos
 - | | |__ Modelos
 - | | | |__ entrenamientos
 - | | |__ Scripts
 - | | | |__ carga_bd
 - | | | |__ clasificadores
 - | | | |__ generacion_datos
 - | | | |__ limpieza_datasets
 - | | | |__ pruebas
 - |__ Reporte

2. Marco Teórico

El marco teórico de este proyecto establece los fundamentos y herramientas que sustentan el desarrollo del sistema propuesto. De esta manera, se abarcan áreas relacionadas con la gestión de inventarios, clasificación de productos, técnicas de ciencia de datos y predicción de demanda mediante aprendizaje automático.

En los siguientes apartados se desarrollan cada uno de estos puntos para facilitar la comprensión del sistema.

2.1. Clasificación de productos

La clasificación de productos es fundamental para la gestión de inventarios, ya que permite organizar los artículos en categorías según sus características, comportamiento de demanda y condiciones de su almacenamiento. Hacer la clasificación facilita la toma de decisiones relacionadas con el control de stock, reposición y su respectivo manejo.

2.1.1. Productos perecederos

Los productos perecederos son aquellos que presentan una vida útil limitada y cuya calidad disminuye con el paso del tiempo por factores estacionales, condiciones climáticas y características propias de cada producto. Este tipo de productos incluye alimentos como frutas, verduras, lácteos y carnes, los cuales requieren condiciones específicas de almacenamiento para conservar sus propiedades.

Según la Organización de las Naciones Unidas para la Alimentación y la Agricultura (FAO) [7], los productos perecederos presentan mayores desafíos en la gestión de inventarios debido a su alta susceptibilidad al deterioro, lo que incrementa las pérdidas por caducidad. Además, su demanda suele ser sumamente variable y sensible a factores externos como la estacionalidad, el clima y las preferencias del consumidor.

2.1.2. Productos no perecederos

Por otro lado, los productos no perecederos son aquellos que poseen una vida útil prolongada y que no presentan un deterioro a corto plazo. Ejemplos de este tipo de productos incluyen alimentos enlatados, productos secos y artículos de limpieza.

En cuestión de inventarios es más fácil gestionar este tipo de productos, debido a que no requieren condiciones especiales de conservación ni un control estricto de caducidad. Sin embargo, su administración sigue siendo relevante, especialmente para evitar problemas de sobrestock o desabasto.

2.1.3. Productos no perecederos

Por otro lado, los productos no perecederos son aquellos que poseen una vida útil prolongada y que no presentan un deterioro a corto plazo. Ejemplos de este tipo de productos incluyen alimentos enlatados, productos secos y artículos de limpieza.

En términos de gestión de inventarios, este tipo de productos tiende a presentar menores riesgos asociados al deterioro, en comparación con los productos perecederos. De acuerdo con la FAO [7], los alimentos como cereales y productos secos registran menores pérdidas a lo largo de la cadena de suministro, debido a su menor susceptibilidad a factores biológicos. No obstante, su correcta administración sigue siendo relevante para evitar problemas de sobrestock o desabasto.

2.1.4. Productos no perecederos

Por otro lado, los productos no perecederos son aquellos que poseen una vida útil prolongada y que no presentan un deterioro a corto plazo. Ejemplos de este tipo de productos incluyen alimentos enlatados, productos secos y artículos de limpieza.

En términos de gestión de inventarios, estos productos presentan una menor complejidad en comparación con los productos perecederos, debido a que tiene menores riesgos asociados al deterioro. De acuerdo con la FAO [7], los alimentos como cereales y productos secos registran menores pérdidas a lo largo de la cadena de suministro, debido a su menor susceptibilidad a factores biológicos. Sin embargo, su administración sigue siendo relevante, especialmente para evitar problemas de sobrestock o desabasto.

2.2. Datos en sistemas de inventario

Los datos en sistemas de inventario es fundamental para tomar decisiones bien informadas, lo que permite analizar el histórico de la demanda, identificar patrones de consumo y optimizar la gestión del stock.

2.2.1. Fuentes de datos

Integrar una mayor diversidad de datos en sistemas de inventario como registros de ventas, movimientos de stock, información de productos, así como datos externos como estacionalidad o eventos que pueden influir en la demanda, permitiendo enriquecer el análisis y mejorar la capacidad predictiva de los modelos.

2.2.2. Preprocesamiento de datos

El preprocesamiento de datos es un proceso crucial en el análisis, ya que los datos suelen contener errores, valores faltantes o inconsistencias. Este proceso consiste en limpiar, transformar y normalizar los datos, con la finalidad de mejorar su calidad y hacerlos funcionales para su posterior análisis.

2.2.3. Ingeniería de características

La ingeniería de características tiene como objetivo crear y transformar variables relevantes a partir de los datos originales, con el propósito de mejorar el desempeño de los modelos predictivos. En el contexto de series de tiempo, es común generar variables como promedios móviles y representaciones cíclicas, que permiten capturar patrones temporales en los datos [8].

2.3. Modelos de Machine Learning para la predicción de demanda

El Machine Learning es una rama derivada de la inteligencia artificial que permite a los sistemas aprender patrones a partir de los datos y realizar predicciones sin ser programados explícitamente para cada tarea.

2.3.1. Modelos de regresión

Los modelos de regresión son técnicas estadísticas y de aprendizaje automático, que se encargan de analizar cómo se relaciona una variable dependiente (variable de salida) y una o más

variables independientes (variables de entrada), con el fin de entender qué factores influyen en un resultado y poder predecir valores futuros.

Mientras que la regresión lineal es clara y permite ver cuanto influye cada factor con el resultado, en el aprendizaje automático se suele sacrificar esa sencillez por modelos mas potentes que, aunque son mas difíciles de explicar, son mucho mas precisos al predecir. [9]

2.3.2. Random Forest

Random Forest combina las predicciones de múltiples árboles de decisión para obtener un resultado más robusto y preciso. Al entrenar cada árbol con una muestra distinta y aleatoria de los datos, el modelo reduce significativamente el riesgo de error y se vuelve sumamente eficaz para manejar grandes volúmenes de información sin presentar sobreajuste.

Es uno de los modelos más utilizados en tareas de clasificación, debido a su buen desempeño en distintos tipos de problemas. Se ha aplicado exitosamente en áreas como la detección de fraudes, el diagnóstico médico basado en síntomas y los sistemas de recomendación en comercio electrónico. Una de sus principales ventajas es que, a pesar de su complejidad, es un modelo que permite identificar qué variables tienen mayor influencia en las decisiones, facilitando así su interpretación. [10]

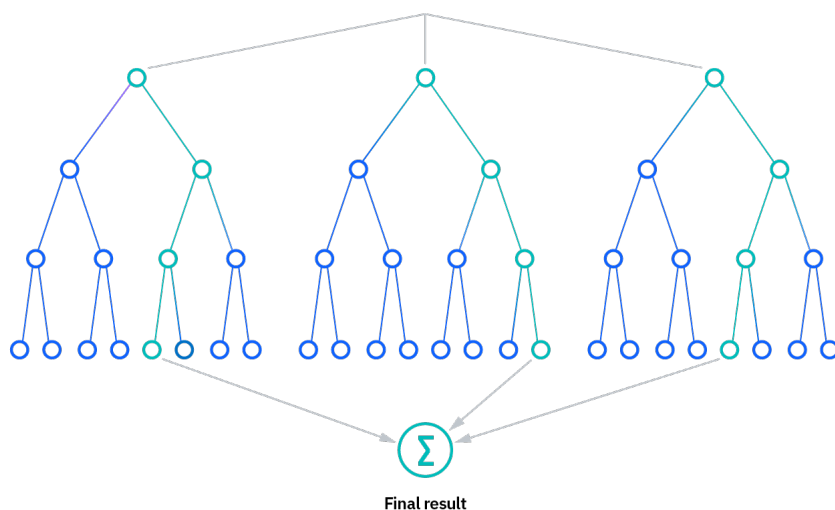


Figura 2.1. Representación del funcionamiento del algoritmo Random Forest

2.3.3. Redes neuronales y evaluación de modelos

Las redes neuronales son modelos de aprendizaje automático inspirados en el funcionamiento del cerebro humano, formados por capas de neuronas artificiales que permiten aprender patrones a partir de los datos. Consiste en transformar un conjunto de entradas en una salida mediante pesos y sesgos que se ajustan durante el entrenamiento, lo que les permite capturar relaciones complejas, especialmente cuando los datos no siguen un comportamiento lineal. Estas redes se estructuran en una capa de entrada, una o varias capas ocultas donde se procesa la información y una capa de salida que genera la predicción, utilizando métodos como el descenso de gradiente y la retropropagación para minimizar el error. Debido a estas características, han sido ampliamente utilizadas en áreas como visión por computadora, procesamiento de lenguaje natural y predicción de series de tiempo, siendo en este proyecto una herramienta adecuada para estimar la demanda a partir de datos históricos de ventas. [11]

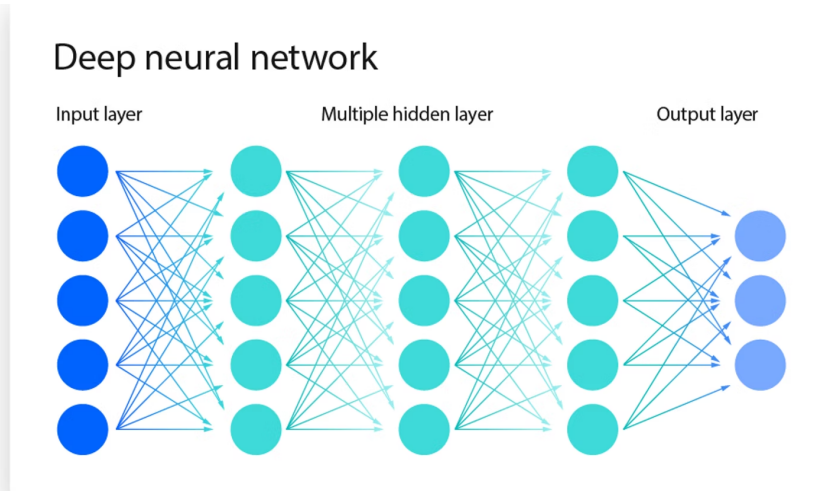


Figura 2.2. Representación de una red neuronal

2.3.4. Evaluación del modelo

Para evaluar el desempeño de los modelos de Inteligencia Artificial, se utilizan diferentes métricas que permiten medir qué tan acertadas son las predicciones respecto a los valores reales.

El Error Absoluto Medio (MAE) representa el error promedio en unidades reales, es decir, indica en promedio cuántas unidades se está equivocando el modelo en sus predicciones. Se calcula como:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (1)$$

donde y_i es el valor real y $\hat{y}_i$ es el valor predicho.

La Raíz del Error Cuadrático Medio (RMSE) también mide el error, pero penaliza más los errores grandes, por lo que es útil para identificar desviaciones importantes. Se define como:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (2)$$

El Error Porcentual Absoluto Ponderado (WAPE) expresa el error en términos porcentuales considerando el total de la demanda, lo que permite entender el impacto global del error:

$$WAPE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n |y_i|} \quad (3)$$

El Error Porcentual Absoluto Medio Simétrico (sMAPE) mide el error relativo de forma balanceada entre valores reales y predichos:

$$sMAPE = \frac{1}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{\frac{|y_i| + |\hat{y}_i|}{2}} \quad (4)$$

Finalmente, el coeficiente de determinación (R^2) indica qué tan bien el modelo explica la variabilidad de los datos:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (5)$$

donde $\bar{y}$ es el valor promedio de los datos reales.

Estas métricas permiten evaluar como se desempeña el modelo desde diferentes perspectivas, facilitando la selección del modelo más acertado.[12]

2.4. Arquitectura de software

La arquitectura de software se refiere a la forma en cómo se organiza un sistema, definiendo cómo interactúan los distintos componentes entre si para cumplir con el objetivo del proyecto. Es fundamental diseñar una buena arquitectura en el desarrollo de aplicaciones, ya que permite estructurar el sistema de manera clara, facilitando su mantenimiento, escalabilidad y adaptación a nuevos requerimientos.

Una buena arquitectura no solo abarca las tecnologías a utilizar, sino también aspectos como la organización del código, la forma en que se comunican los módulos entre sí y la capacidad del sistema para crecer sin afectar su funcionamiento. Existen patrones de arquitectura previamente diseñados, como el modelo de capas, el basado en eventos o el de microservicios [13]; sin embargo, lo más importante es considerar las necesidades que se buscan resolver para seleccionar la arquitectura más adecuada, de modo que sea eficiente y no solo funcional, adaptándose según lo requerido por el problema.

2.4.1. Modelo MVC

El modelo MVC (Modelo-Vista-Controlador) es un patrón de diseño que divide una aplicación en tres componentes principales[14]:

- **Modelo:** Es el encargado de representar los datos y gestionar la lógica del negocio. Su función principal es acceder a la base de datos, aplicar reglas de negocio, realizar validaciones y procesar la información. Es importante recalcar que en ningún momento se comunica directamente con la vista.
- **Vista:** Su función es mostrar la información que proporciona el modelo a través de una interfaz gráfica o página web. Su responsabilidad es únicamente visual, es decir, presentar los datos en un formato adecuado, sin incluir lógica de negocio.
- **Controlador:** Gestiona las acciones del usuario y decide qué datos se deben consultar y cómo deben mostrarse. Recibe la solicitud, interactúa con el modelo para obtener los datos y selecciona la vista adecuada para generar la respuesta.

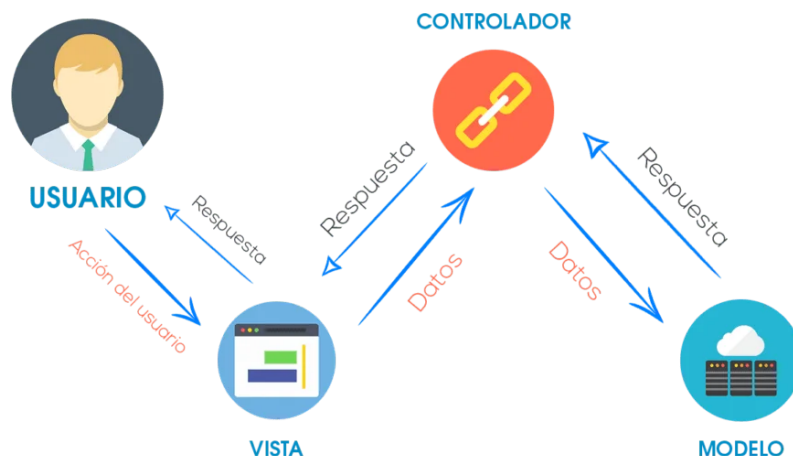


Figura 2.3. Interacción del modelo MVC

2.4.2. Backend con FastAPI

Una API (Interfaz de Programación de Aplicaciones) es un conjunto de reglas que permite la comunicación entre distintos sistemas o componentes de software, facilitando el intercambio de datos mediante solicitudes y respuestas[15].

FastAPI es un framework utilizado en el desarrollo de aplicaciones web con Python, basado en la creación de APIs. Su arquitectura se fundamenta en el uso de servicios y rutas (endpoints), lo que permite organizar la lógica del sistema en módulos independientes. Facilitando la validación de datos, la integración con bases de datos y el desarrollo de aplicaciones escalables[16].

2.4.3. API REST

Una API REST es una interfaz de programación que permite la comunicación entre diferentes sistemas mediante solicitudes HTTP (Protocolo de Transferencia de Hipertexto), siguiendo la arquitectura REST (Transferencia de Estado Representacional). Se basa en el modelo cliente-servidor sin estado. Cada solicitud es independiente y contiene la información necesaria para ser procesada, facilitando su uso y escalabilidad. Para la transferencia de la información, usualmente se utilizan formatos ligeros y estandarizados como JSON; es el más común, aunque no el único utilizado, permitiendo una integración sencilla entre aplicaciones[17].

2.5. Desarrollo frontend con React

React es una biblioteca de JavaScript que se utiliza para construir interfaces de usuario interactivas, basada en el uso de componentes reutilizables. Permite desarrollar aplicaciones web dinámicas, donde la interfaz se actualiza en respuesta a cambios en los datos[18].

3. Desarrollo del proyecto

En este capítulo se describe el proceso de construcción del sistema "Modelo inteligente para la predicción de inventario de productos sensibles al tiempo en microempresas". Se abordan elementos como el diseño y la construcción de modelos de machine learning, la construcción de una base de datos relacional en MySQL y la integración con una aplicación web completa (frontend y backend) para el uso sencillo e intuitivo del sistema desarrollado. La figura 3.1 representa gráficamente los módulos del sistema, los cuales incluyen: (1) Módulo de preparación y entrenamiento del modelo, tratamiento de los datasets originales y los obtenidos de fuentes externas; (2) Módulo backend y servicios, lógica del negocio e interacción con la base de datos; (3) Módulo frontend e interacción con usuarios, panel gráfico de la aplicación y el único módulo con el que el usuario general (empleado) tiene interacción.

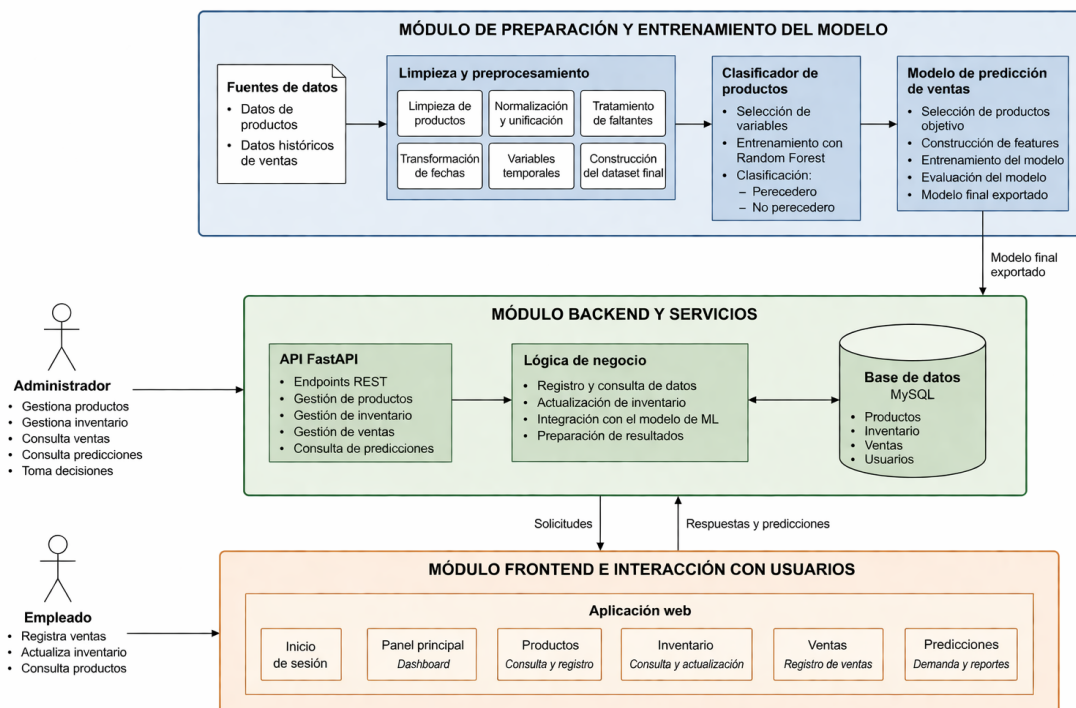


Figura 3.1. Esquema general del sistema Modelo inteligente para la predicción de inventario de productos sensibles al tiempo en microempresas

3.1. Obtención de datasets

En el desarrollo inicial del proyecto no se contaba con un dataset estructurado de productos comunes en un comercio local (frutas, verduras y productos correspondientes a una tienda), por lo que se recurrió al uso de herramientas de inteligencia artificial (ChatGPT Plus) para generar un catálogo base de productos. Este catálogo incluye atributos relevantes como product_name (nombre del producto), category_off (categoría a la que pertenece el producto) y los días de vida útil en diferentes tipos de almacenamiento: en despensa (temperatura ambiente), en refrigeración y en congelamiento.

La herramienta de IA proporcionó algunas características adicionales de cada producto, como barcode (código de barras), marca y país en el que se comercializa; sin embargo, para fines de este

proyecto, estos atributos se descartaron debido a que no aportan información relevante para los objetivos planteados, como se muestra en el fragmento 1.

```
{
  "barcode": "7501234567897",
  "brand": "Jumex",
  "product_name": "Nectar de durazno 1 L",
  "category_off": "juice-box",
  "shelf_life_pantry_days": 270,
  "shelf_life_fridge_days": 10,
  "shelf_life_freezer_days": 60,
  "source_rule": "Heuristic: Juice boxed unopened 6-9 m; opened fridge 7-10 d",
  "country": "Mexico",
  "confidence": 0.55,
  "generated_on": "2025-10-08"
}
```

Fragmento 1: Ejemplo de la estructura generada por la herramienta de inteligencia artificial.

Dado que el dataset inicial de productos, generado mediante la herramienta de inteligencia artificial, presentaba inconsistencias en los nombres y diferencias entre productos, y considerando que el dataset original de ventas únicamente contenía registros sin características adicionales relevantes, como el tiempo de vida útil en distintos escenarios de almacenamiento, se concluyó que la información recolectada a hasta este momento era insuficiente para el entrenamiento de un modelo de red neuronal. Por lo que se investigó sobre la fiabilidad y utilidad de los datos generados por este tipo de herramientas de IA. A partir de este análisis, se optó por utilizar la inteligencia artificial como una herramienta de apoyo para analizar el dataset original de ventas y construir un nuevo conjunto de datos más estructurado, consistente y con un mayor número de registros. Lo que permitió enriquecer significativamente la información disponible, incorporando características relevantes para el problema y mejorando la calidad del dataset, lo que también facilitó el proceso de modelado y aceleró el desarrollo del proyecto.

```
fecha,product_name,category_off,ventas,precio
2022-12-01,Nectar de durazno 1 L,juice-box,10,33.72
2022-12-01,Arroz blanco 500 g,rice-white-dry,3,45.84
2022-12-01,Betabel,fruits-vegetables,33,30.22
```

Fragmento 2: Ejemplo del dataset de ventas en formato CSV.

El uso de herramientas de inteligencia artificial para la generación de datos sintéticos ha sido explorado en distintos contextos como una alternativa viable cuando no se dispone de información real suficiente [19], por lo que su aplicación en este proyecto se justifica como un mecanismo para complementar y fortalecer los datos utilizados en el proceso de entrenamiento del modelo predictivo.

3.2. Preprocesamiento y clasificación de productos

Una vez obtenido el catálogo principal de productos, se llevó a cabo un proceso de limpieza, normalización, relleno de valores nulos y transformación de los datos, con el fin de que el modelo de clasificación fuera capaz de procesar la información. Posteriormente, con los datos ya preparados, se implementó un modelo de aprendizaje automático basado en el algoritmo de *Random*

Forest, el cual permitió clasificar los productos en dos categorías: perecederos y no perecederos (Véase en Apéndice A.1: Código fuente del módulo clasificador_perecedero.py).

Realizar esta clasificación es fundamental para continuar con el desarrollo del sistema, ya que permite diferenciar el comportamiento de los productos y su impacto en la gestión del inventario.

3.3. Obtención de datos de eventos y festividades

Para hacer más precisa la predicción de la demanda, se contemplaron factores externos como eventos y festividades relevantes en la Ciudad de México. Para cumplir con este objetivo, se desarrolló un script en Python para obtener un dataset con esta información (Véase en Apéndice A.2: Código fuente del módulo obtener_feriados_CDMX.py), como se muestra en el fragmento 3. Para ello, se utilizó la API *Calendarific*, la cual proporciona información estructurada sobre días festivos a nivel internacional [20].

La inclusión de estos datos permite considerar variaciones en el comportamiento de la demanda asociadas a fechas específicas.

```
date,event,scope
2026-01-01,New Year's Day,National holiday
2026-01-06,Day of the Holy Kings,Observance
2026-02-02,Candlemas,Observance
```

Fragmento 3: Ejemplo del dataset de eventos proporcionados por la API de Calendarific

3.4. Integración de temporadas de productos perecederos

Se integro un dataset de productos perecederos, específicamente frutas y verduras, el cual contiene información sobre su temporada pico, se muestra en el fragmento 4, es decir, el periodo del año en el que estos productos presentan mayor disponibilidad en el mercado debido a su estacionalidad.

Posteriormente, esta información fue normalizada con la finalidad de tener variables estructuradas (Véase en Apéndice A.3: Código fuente del módulo normalizacion_fruits-vegetables_temporadas.py), como un indicador de disponibilidad durante todo el año, así como los meses de inicio y fin de la temporada pico de cada producto. Se representa en el fragmento 5

```
nombre,tipo,temporada_pico
Aguacate,Fruta,Dic-May (alta); disponible todo el año
Ciruela,Fruta,Jul-Ago
Durazno,Fruta,Jul-Sep
Fresa,Fruta,May-Jun
```

Fragmento 4: Ejemplo del dataset de productos perecederos

```
,nombre,tipo,temporada_pico,todo_el_anio,temporada_pico_inicio,temporada_pico_fin
0,Aguacate,fruits-vegetables,Dic-May (alta); disponible todo el año,True,12.0,5.0
1,Ciruela,fruits-vegetables,Jul-Ago,False,7.0,8.0
2,Durazno,fruits-vegetables,Jul-Sep,False,7.0,9.0
3,Fresa,fruits-vegetables,May-Jun,False,5.0,6.0
```

Fragmento 5: Ejemplo del dataset de productos perecederos normalizado

3.5. Generación de dataset de eventos asociados a productos

A partir de la integración de los datasets obtenidos previamente (eventos, temporadas de productos y productos clasificados), se construyó un nuevo conjunto de datos que relaciona los eventos con productos y sus respectivas fechas, fragmento 6 (Véase en Apéndice A.4: Código fuente del módulo generar_eventos_productos.py).

Este dataset permite relacionar eventos con la demanda de ciertos productos, lo cual es relevante para mejorar la capacidad predictiva del sistema.

```
event,date,producto_relacionado
New Year's Day,2026-01-01,Aguacate Hass
New Year's Eve,2026-12-31,Agua mineral 600 ml
New Year's Eve,2026-12-31,Tejocote
```

Fragmento 6: Ejemplo del dataset de productos relacionados con su respectiva fecha

3.6. Normalización del dataset de ventas

Con los datasets obtenidos en las etapas anteriores, se llevo a cabo una integración y normalización para el conjunto de datos del histórico de ventas (Véase en Apéndice A.5: Código fuente del módulo completar_dataSet_ventas.py) . Como resultado, se obtuvo un dataset mas optimo para el modelo de predicción, debido a que incluye variables relevantes como indicadores de temporalidad y rangos de temporada, como se muestra en el fragmento 7.

Esta estructura permite integrar características propias del producto como factores externos que influyen en su comportamiento de venta, lo que facilita entender mejor la demanda.

```
, fecha,product_name,category_off,ventas,precio,perecedero,en_temporada,temp_
  inicio_mes,temp_fin_mes
5,2022-12-01,arroz blanco 500 g,rice-white-dry,3,45.84,0.0,1,1.0,12.0
6,2022-12-01,betabel,fruits-vegetables,33,30.22,1.0,1,1.0,12.0
25,2022-12-01,refresco limon 600 ml,soft-drinks,2,28.75,0.0,1,1.0,12.0
```

Fragmento 7: Ejemplo del dataset de ventas normalizado

3.7. Entrenamiento del modelo predictivo

Con los conjuntos de datos previamente tratados y normalizados, se procedió a hacer el entrenamiento del modelo predictivo de demanda. Para esta tarea, se integraron dos fuentes principales: el dataset de ventas y el dataset de eventos asociados a los productos, permitiendo relacionar el comportamiento histórico de ventas con factores externos.

A partir de esta integración, se generaron distintas variables de apoyo para el modelo, incluyendo características temporales, indicadores de eventos, codificaciones cíclicas (mes y día de la semana), así como variables basadas en el historial de ventas, tales como rezagos (*lag*) y promedios móviles en diferentes ventanas de tiempo.

Previo al entrenamiento, el dataset fue dividido en subconjuntos de entrenamiento, validación y prueba, utilizando un enfoque basado en fechas para respetar la naturaleza temporal de los datos. Esto permitió evaluar el desempeño del modelo sobre datos no vistos.

Posteriormente, se implementó un modelo basado en redes neuronales utilizando la librería *TensorFlow/Keras*. La arquitectura del modelo consiste en varias capas densas completamente conectadas, con funciones de activación ReLU y capas de regularización mediante *Dropout*, lo que contribuye a reducir el sobreajuste.

Durante el entrenamiento, se utilizaron técnicas como *Early Stopping* y ajuste dinámico de la tasa de aprendizaje, con el objetivo de mejorar la estabilidad del modelo y su capacidad de generalización.

Finalmente, el desempeño del modelo fue evaluado mediante métricas como el error absoluto medio (MAE), la raíz del error cuadrático medio (RMSE) y el error porcentual absoluto medio (MAPE), permitiendo cuantificar la precisión de las predicciones generadas (Véase en Apéndice A.6: Código fuente del módulo `entrenamiento_modelo_demanda.py`).

3.8. Diseño de la base de datos

En un sistema informático orientado a la gestión de stock y control de ventas, es de vital importancia diseñar un componente de base de datos que se encargue de garantizar la integridad, disponibilidad y seguridad de la información. En este proyecto, la base de datos es responsable de guardar los productos disponibles en el almacén, registrar todos los movimientos del inventario (entradas y salidas) y guardar cada una de las ventas realizadas.

Para este proyecto, y debido a que la complejidad de la base de datos no es demasiado alta, desde un inicio se planteó el uso de una base de datos relacional con MySQL. Esto se debe a que la información presenta una estructura bien definida y el sistema no requiere escalabilidad horizontal, por lo que un sistema relacional resulta la mejor opción.

3.8.1. Modelo relacional

En la figura 3.2 se muestra el modelo entidad-relación correspondiente a la base de datos utilizada por el sistema backend de la aplicación. Este modelo permite mantener la integridad referencial y facilitar operaciones típicas como registrar, actualizar y eliminar productos en el almacén, registrar nuevas ventas y ajustar el stock que hay por producto.

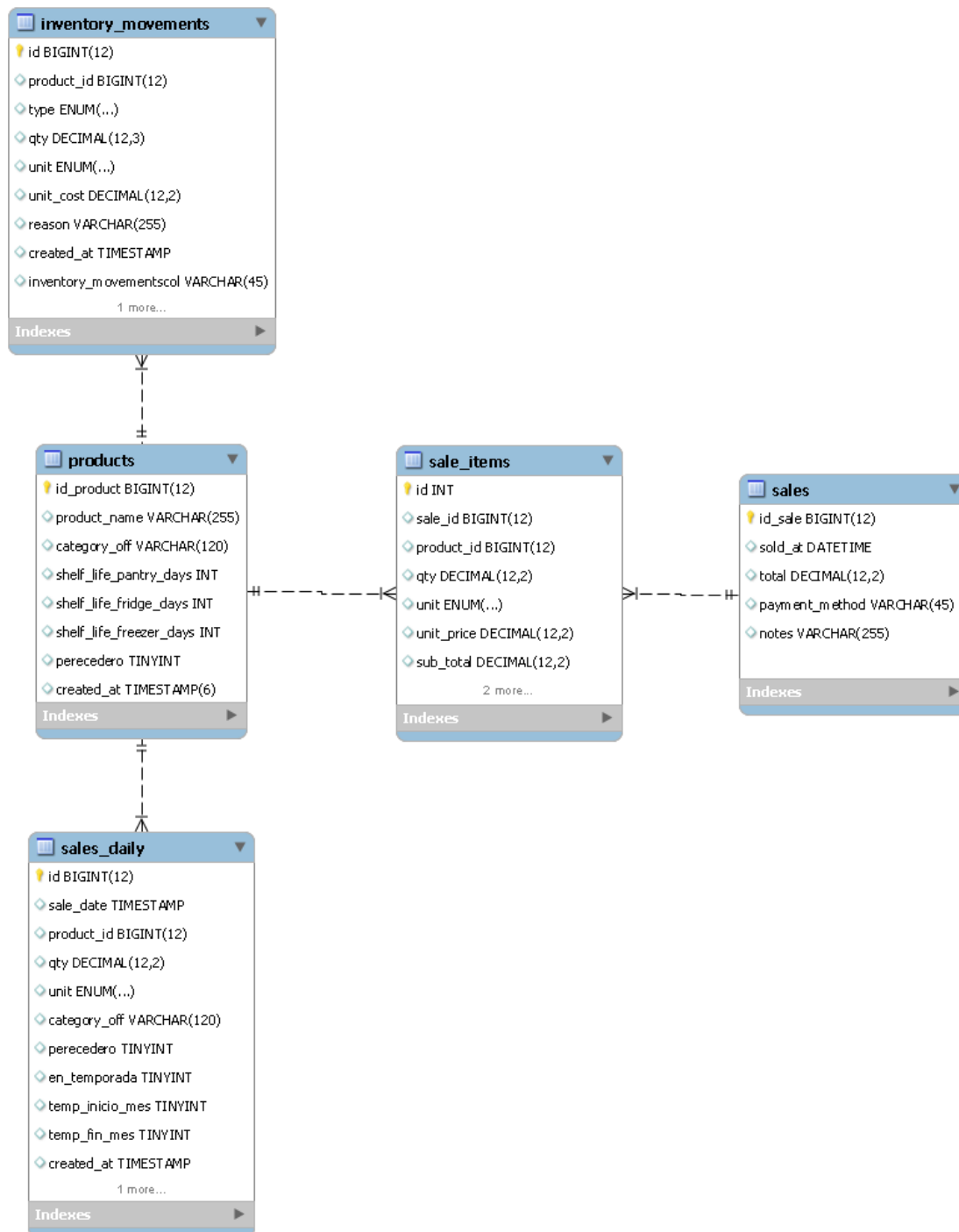


Figura 3.2. Modelo relacional de la base de datos

3.8.2. Estructura de tablas principales

El modelo propuesto contempla las siguientes entidades clave:

- **productos**: Tabla principal encargada de almacenar la información general de los productos. Incluye atributos como el nombre, la categoría a la que pertenece y su vida útil expresada en

días bajo distintos tipos de almacenamiento: en almacén, en refrigeración y en congelación.

- **inventory_movements:** Se encarga de almacenar todos los movimientos que se realizan en el almacén, ya sean ventas (salidas) o entradas (reabastecimiento). Incluye datos clave como el tipo de movimiento, definido como un *ENUM('IN', 'OUT', 'ADJUST')*, la cantidad movida, la unidad de medida ('kg', 'g', 'piece', 'pack', 'box', 'lt', 'ml'), una razón del movimiento y la fecha en que se realizó.
- **sales:** Entidad que almacena la información general de cada transacción de venta. Incluye atributos como la fecha de la operación, el método de pago utilizado, notas adicionales y el monto total de la venta.
- **sale_items:** Tabla que descompone cada venta en los productos individuales que la conforman. Registra información como la cantidad vendida por producto, la unidad de medida ('kg', 'g', 'piece', 'pack', 'box', 'lt', 'ml') y el subtotal correspondiente. Esta estructura permite un análisis detallado del comportamiento de venta por producto.
- **sales_daily:** Entidad que almacena ventas diarias por producto. Presenta una estructura similar al fragmento 5 y tiene como objetivo facilitar la exportación de datos para procesos posteriores, como el reentrenamiento del modelo de predicción. Esto permite adaptar el sistema a las particularidades de cada negocio que lo implemente.

Este modelado permite la persistencia de la información de forma estructurada, lo que facilita la ejecución eficiente de consultas desde el backend. Contribuye a mantener la integridad y consistencia de los datos, lo cual es fundamental para la correcta operación del sistema.

Adicionalmente, la estructura propuesta favorece la escalabilidad del proyecto en caso de que el negocio experimente un crecimiento, ya que permite incorporar nuevas entidades, relaciones o atributos sin afectar significativamente el funcionamiento existente. De igual forma, sienta las bases para la integración con otros módulos, como los sistemas de análisis de datos y los modelos de predicción.

El script completo para la creación de las tablas DDL se visualiza Apéndice A.7: Lenguaje de Definición de Datos scriptBD.mysql

3.9. Implementación del backend

Para el desarrollo del backend, que es el encargado de interactuar con la base de datos, realizar las validaciones correspondientes a la lógica del negocio y exponer la información mediante endpoints, se utilizó el framework web moderno FastAPI.

Este framework fue seleccionado debido a su alto rendimiento, facilidad de uso y su integración nativa con tipado estático mediante Python, lo que permite desarrollar APIs robustas y bien estructuradas. FastAPI facilita la validación automática de datos mediante esquemas definidos, así como la generación automática de documentación interactiva a través de herramientas como Swagger [21].

El backend se encarga de gestionar las operaciones principales del sistema, tales como el registro y consulta de productos, la gestión de ventas, el control de movimientos de inventario y la comunicación con el módulo de predicción. Además, implementa una arquitectura modular que organiza el código en componentes como modelos, esquemas, servicios y rutas, lo cual mejora la mantenibilidad y escalabilidad del sistema.

3.9.1. Estructura del sistema

El backend de la aplicación web fue desarrollado siguiendo una arquitectura modular, apoyada en principios de diseño como SOLID, con el objetivo de tener un código organizado, mantenible y escalable. Este se expone mediante una API REST, lo que permite comunicarse con otros sistemas como el frontend. Esta estructura permite que cada componente tenga un propósito único y bien definido, dando como resultado un sistema organizado, fácil de mantener y con la capacidad de escalar ante futuras actualizaciones.

En primer lugar se tiene el módulo de *models*, el cual se encarga de definir las entidades de la base de datos mediante el uso de un ORM (Object-Relational Mapping o Mapeo Objeto-Relacional), el cual se encarga de mapear las tablas relacionales a objetos dentro del código, véase el código fuente del módulo en el apéndice A.8.1. Por otro lado, está el módulo de *schemas*, es el encargado de definir los esquemas de validación de datos utilizando tipado estático, lo cual asegura la integridad de la información recibida y se envía a través de la API, véase el código fuente del módulo en el apéndice A.8.2.

El módulo de *routers* funciona como el punto de acceso a la aplicación, donde se definen los *endpoints* del sistema, véase el código fuente del módulo en el apéndice A.8.3. Mientras que en el módulo de *services* se encapsula la lógica del negocio, lo que permite separar las validaciones y procesos internos del sistema de la capa encargada de exponer la API. Esta separación facilita la reutilización de código y mejora la mantenibilidad del sistema, véase el código fuente del módulo en el apéndice A.8.4.

3.9.2. Integración del modelo predictivo

El modelo predictivo se integró con el backend reutilizando los modelos entrenados previamente. De esta manera, el sistema no realiza el entrenamiento en tiempo de ejecución, sino que carga modelos ya entrenados para su uso en la fase de predicción.

El flujo de este módulo consiste en recibir una solicitud a través del endpoint correspondiente, procesar los datos de entrada para darles el formato requerido por el modelo y posteriormente generar la predicción de demanda. El backend actúa como una capa intermedia entre el usuario y el modelo de aprendizaje automático, lo que permite controlar las entradas, validar la información y estandarizar las salidas del sistema. El código fuente de la implementación de los modelos y su integración en el backend vease en el apéndice A.9

3.10. Implementación del frontend

El frontend de la aplicación web fue diseñado con el objetivo de que el usuario final tenga una interacción sencilla e intuitiva. Debido a que el sistema está dirigido a microempresas, se buscó que la interfaz gráfica permitiera consultar información relevante, capturar datos y visualizar predicciones sin requerir conocimientos técnicos avanzados. Se muestra la información proveniente del backend, este módulo es el encargado de gestionar la navegación entre las diferentes secciones, el envío de formularios, validación básica de entradas y presentar resultados de forma comprensible. De este modo, se convierte en la capa de interacción directa entre el usuario y la lógica del sistema.

3.10.1. Arquitectura general del frontend

Para el desarrollo del frontend se utilizó la biblioteca de JavaScript *React*, la cual permite desarrollar interfaces de usuario con componentes reutilizables. Esta herramienta facilita la creación de

aplicaciones web. El uso de React permitió construir componentes independientes, lo que facilitó la reutilización de código y una mejor organización del proyecto. Asimismo, se utilizó *TypeScript*, una extensión de JavaScript que incorpora tipado estático, lo que permite reducir errores durante el desarrollo y mejorar la claridad del código.

Para la navegación entre las diferentes secciones de la aplicación, se implementó un sistema de rutas que permite acceder a cada una de las interfaces mediante una URL específica dentro del sistema. La gestión de esta navegación se realizó con un menú lateral, lo que facilita un acceso estructurado y eficiente.

La comunicación con el backend se realiza mediante solicitudes HTTP utilizando funciones asíncronas, lo que permite obtener y enviar información sin interrumpir la interacción del usuario con la interfaz.

La arquitectura del frontend se compone de los siguientes elementos:

- **Páginas:** representa las pantallas principales del sistema, como el tablero general, la gestión de productos, el registro de ventas y la sección de predicciones.
- **Componentes reutilizables:** encapsula elementos visuales y funcionales que se emplean en distintas partes de la aplicación.
- **Servicios de comunicación:** permite establecer la conexión con el backend para consultar, registrar o actualizar información mediante solicitudes HTTP.
- **Estilos:** Especifica la representación visual del sistema, manteniendo uniformidad en colores, tamaños y espaciados.

4. Resultados

Este capítulo presenta los resultados obtenidos tras la implementación e integración completa de todos los módulos descritos previamente. Se documenta el comportamiento del sistema durante las pruebas funcionales y de validación, tanto a nivel modular como en escenarios de uso integrados.

4.1. Evaluación del modelo predictivo

El desempeño del modelo predictivo fue evaluado con diferentes métricas que permiten analizar su precisión.

En primer lugar, el **error absoluto medio (MAE)** obtuvo un valor de **2.84 unidades**, lo que significa que, en promedio, las predicciones del modelo tienen una desviación aproximada de tres unidades con respecto a las ventas reales. Este resultado se puede considerar bajo, específicamente en productos con demanda constante, ya que permite aproximar cantidades de reabastecimiento con un abajo margen de error.

Otra métrica de evaluación fue la **raíz del error cuadrático medio (RMSE)** con un valor de **8.95 unidades**, un valor superior al MAE, lo que indica la presencia de errores más grandes en algunos puntos específicos. Este comportamiento es común en problemas de predicción de demanda, donde existen variaciones asociadas a la estacionalidad o a eventos particulares que afectan las ventas de ciertos productos, haciendo que el modelo no logre comprender y generalizar completamente estos patrones.

Esto se debe, en gran medida, a que los productos perecederos tienen una demanda altamente dependiente de su temporalidad, ocasionando grandes variaciones en su comportamiento de venta. Estas alteraciones hacen difícil que el modelo identifique y generalice estos patrones, haciendo que la predicción sea más compleja y propensa a errores.

En métricas relativas, el modelo presentó un **WAPE (Weighted Absolute Percentage Error) del 26.96 %**, lo que indica que el error fue aproximadamente el 27 % del total de las ventas. Este porcentaje de error se encuentra dentro de un valor aceptable para aplicaciones reales de pronóstico, donde la inestabilidad de la demanda hace difícil alcanzar niveles de precisión más bajos. Por otro lado, el **sMAPE (Symmetric Mean Absolute Percentage Error) fue de 34.11 %**, lo que confirma la existencia de variaciones entre los valores reales y predichos, especialmente en escenarios de baja o alta demanda.

Finalmente, el modelo obtuvo un **coeficiente de determinación (R^2) de 0.76**, lo que significa que es capaz de describir aproximadamente el 76 % de la variabilidad en las ventas. Este resultado es relevante, debido a que indica que el modelo no genera predicciones aleatorias, sino que ha logrado encontrar patrones relevantes dentro de los datos históricos.

4.2. Justificación del desempeño del modelo

El valor de R^2 obtenido puede considerarse adecuado para el contexto del problema abordado. En sistemas de predicción de demanda, especialmente en microempresas y productos perecederos, el comportamiento de las ventas suele estar influenciado por múltiples factores difíciles de modelar completamente, como cambios en el clima, variaciones en la preferencia del consumidor, promociones no registradas o eventos locales.

Bajo estas condiciones, alcanzar un nivel de explicación cercano al 75 % representa un resultado sólido, ya que implica que el modelo captura la mayor parte de la dinámica de las ventas, manteniendo un equilibrio entre precisión y capacidad de generalización. Intentar aumentar significativamente este valor podría llevar a un sobreajuste, reduciendo la capacidad del modelo para adaptarse a datos nuevos.

El valor de R^2 obtenido se considera aceptable para este contexto, ya que en la predicción de demanda, especialmente en productos perecederos, las ventas se ven influenciadas por diferentes factores difíciles de modelar, como el clima, las preferencias del consumidor o eventos no registrados. En estas condiciones, alcanzar un nivel cercano al 75 % indica que el modelo logra comprender la mayor parte del comportamiento de las ventas, manteniendo un buen equilibrio entre precisión y capacidad de generalización, evitando así problemas de sobreajuste.

4.3. Evaluación del backend

A diferencia del modelo predictivo, cuyo desempeño se evalúa mediante métricas cuantitativas, el backend se evalúa con una perspectiva funcional, considerando su capacidad para manejar la lógica del negocio, la integración de los componentes y la respuesta a solicitudes. Desde esta perspectiva, el sistema backend cumple con los requerimientos necesarios, permitiendo manejar la gestión de productos, el registro de ventas, el control de inventario y la generación de predicciones, lo cual fue validado mediante pruebas sobre los distintos endpoints, verificando la coherencia de las respuestas con los datos almacenados.

De este modo, se comprobó la correcta integración entre los módulos del sistema, en donde el backend actúa como intermediario entre la base de datos, el modelo de aprendizaje automático y el frontend, garantizando una comunicación continua entre los módulos. Además, el uso de FastAPI permite un manejo eficiente de solicitudes, demostrando un comportamiento estable. Finalmen-

te, la arquitectura modular facilita la mantenibilidad y escalabilidad del sistema, permitiendo su evolución sin afectar sus componentes.

4.4. Implementación del frontend

4.4.1. Vistas principales del sistema

Se desarrollaron diferentes vistas para la interfaz web, diseñadas para cubrir las funcionalidades principales del sistema. El código fuente de estas vistas se encuentra en la carpeta descrita en la sección 1.5.2. En este apartado se describen las pantallas más relevantes y su función dentro de la aplicación.

Pantalla principal o dashboard En la vista inicial del sistema se presenta un resumen general de la información relacionada con las ventas diarias, mostrando indicadores como el monto total vendido del día actual, el número de tickets generados y el total de productos vendidos. Además, se incorpora una pequeña gráfica que permite visualizar la tendencia de las ventas en días anteriores, considerando la cantidad de productos vendidos durante ese periodo. Esta vista permite al usuario tener una visión rápida de las métricas diarias del negocio, sin necesidad de navegar por otras secciones de la aplicación. Esta vista se muestra en la figura 4.1

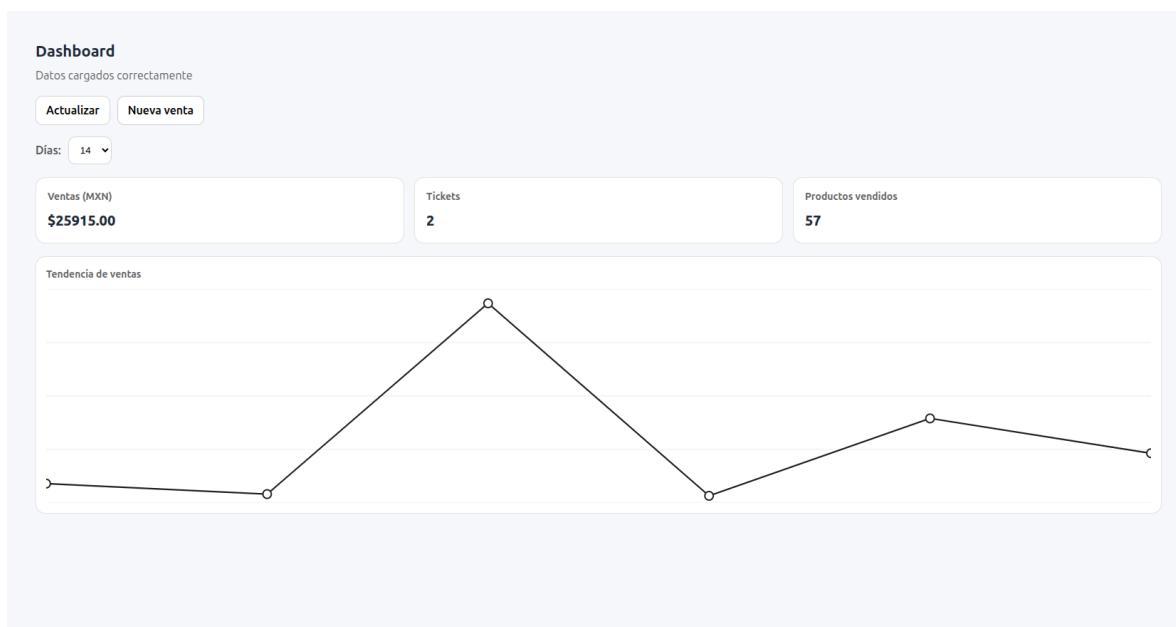


Figura 4.1. Vista del dashboard principal del sistema.

Pantalla de productos En la vista de productos se puede consultar el catálogo disponible en el sistema. Mostrando los productos almacenados con sus atributos principales, facilitando su revisión y administración.

Productos 148 registros

Buscar Producto:

PRODUCTO	CATEGORÍA	PERECEDERO	ACCIONES
durazno	juice-box	si	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
sardinas en salsa 155 g	canned-fish	no	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
queso gouda 300 g	cheese-hard	si	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
frijol pinto 1 kg	beans-dry	no	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
huevo blanco 12 pzas	eggs	si	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
azucar blanca 1 kg	sugar-white	no	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
frijoles refritos 430 g	canned-beans	si	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
sal fina 750 g	salt	no	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
tortillas de maiz 1 kg	tortillas-corn	si	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
aguacate hass	avocado	si	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
queso cheddar 200 g	cheese-hard	si	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
helado vainilla 1 l	ice-cream	si	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>
azucar estandar 2 kg	sugar-white	no	<button>Ver</button> <button>Editar</button> <button>Eliminar</button>

Figura 4.2. Vista de gestión y consulta de productos.

En esta pantalla también se gestionan las operaciones de búsqueda, edición o actualización de productos. Se encarga de mantener la información accesible para el usuario.

Dentro de esta vista, la opción **Ver** permite desplegar una alerta que muestra toda la información del producto seleccionado, como se representa en la figura 4.3, facilitando una revisión rápida de sus atributos sin necesidad de cambiar de pantalla. Por otro lado, la opción **Editar** muestra una ventana emergente donde se pueden modificar los datos del producto, como su nombre, categoría y valores relacionados con su vida útil, permitiendo actualizar la información de manera directa dentro del sistema, se puede ver en la figura 4.4.

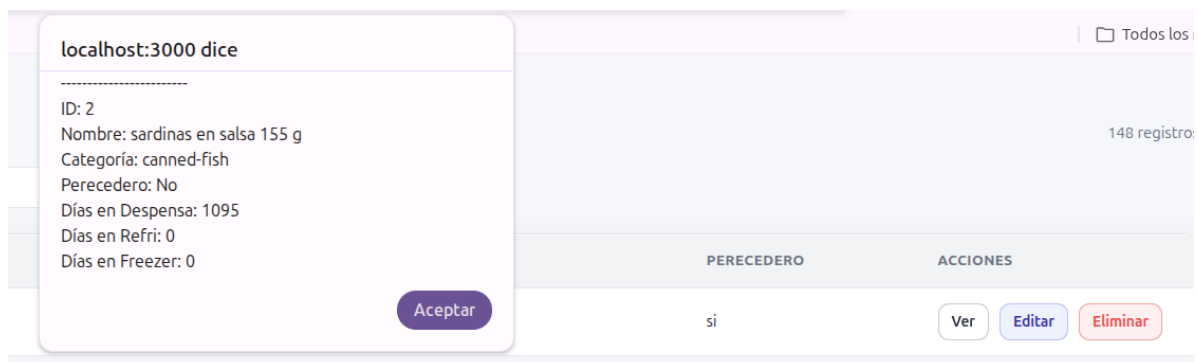


Figura 4.3. Alerta de visualización de un producto.

Editar producto

×

Actualiza los campos y guarda los cambios.

Nombre	Categoría
sardinas en salsa 155 g	canned-fish
Vida útil en Despensa	Vida útil en Refrigeración
1095	0
Vida útil en Congelador	
0	

Preview JSON

Solo lectura

```
{  "product_name": "sardinas en salsa 155 g",  "category_off": "canned-fish",  "shelf_life_pantry_days": 1095,  "shelf_life_fridge_days": 0,  "shelf_life_freezer_days": 0,  "id": 2,  "perecedero": 0}
```

Figura 4.4. Edición de un producto.

Pantalla de registro de ventas La vista de ventas está diseñada para capturar las transacciones realizadas, permite relacionar productos, cantidad de producto y valor unitario correspondientes. Facilita el registro de la información que posteriormente es utilizada por el sistema para el control del inventario.

Nueva venta

Método de pago

Notas

Efectivo

Venta en mostrador

PRODUCTO

CANTIDAD

UNIDAD

PRECIO

SUBTOTAL

ACCIÓN

mamey - fruits-vegetables

03

Kg

050

150

Quitar

sardinas en salsa 155 g - canned-fish

03

Pieza

33.50

100.5

Quitar

trijol pinto 1 kg - beans-dry

05

Pieza

027.50

137.5

Quitar

Total

\$388

Artículos

3

JSON hacia el backend

```
{
  "items": [
    {
      "product id": 142,
      "qty": 3,
      "unit": "kg",
      "unit_price": 50
    },
    {
      "product id": 2,
      "qty": 3,
      "unit": "piece",
      "unit_price": 33.5
    },
    {
      "product id": 4,
      "qty": 5,
      "unit": "piece",
      "unit_price": 27.5
    }
  ]
}
```

Borrar

Guardar

Figura 4.5. Vista para el registro de ventas.

Esta pantalla es relevante debido a que permite alimentar a la base de datos con la información diaria del negocio.

Historial de ventas Esta vista permite consultar los registros de ventas almacenados dentro del sistema. Muestra información como la fecha, el producto vendido, la cantidad, el precio y el total correspondiente a cada registro, facilitando su revisión y seguimiento.

Historial de ventas

Listo: 11 fila(s).

[Nueva venta](#) [Exportar CSV](#)

Desde: dd/mm/aaaa Hasta: 28/03/2026

[Limpiar](#) [Aplicar filtros](#)

Fecha	Producto	Cantidad	Precio	Total
2026-03-19	azucar estandar 2 kg	20.00 kg	\$17.00	\$340.00
2026-03-19	azucar estandar 2 kg	15.00 kg	\$18.00	\$270.00
2026-03-19	azucar estandar 2 kg	5.000 piece	\$36.50	\$182.50
2026-03-19	azucar estandar 2 kg	3.000 piece	\$36.50	\$109.50
2026-03-18	calabacita	56.00 box	\$1999.00	\$111944.00
2026-03-17	toronja pomelo	20.00 piece	\$25.50	\$510.00
2026-03-17	arroz blanco 500 g	15.20 piece	\$15.23	\$231.50
2026-03-17	sardinas en salsa 155 g	4.000 piece	\$231.16	\$924.62
2026-03-17	durazno	15.00 piece	\$15.21	\$228.15
2026-03-13	frijol pinto 1 kg	15.00 kg	\$30.00	\$450.00
2026-03-13	durazno	15.00 kg	\$500.00	\$7500.00

Figura 4.6. Vista de consulta del historial de ventas.

Esta pantalla también puede filtrar los registros dado un rango de fechas específico, facilitando la búsqueda de ventas realizadas en un periodo determinado. También incorpora la opción de registrar una nueva venta y exportar la información mostrada en formato CSV, otorgando un mejor manejo de los datos.

Pantalla de predicciones La vista de predicciones es una de las secciones más relevantes del sistema, ya que permite consultar la predicción de demanda por cada producto en un periodo establecido. En esta pantalla el usuario puede seleccionar los parámetros necesarios y obtener un resultado generado por el modelo predictivo integrado en el backend.

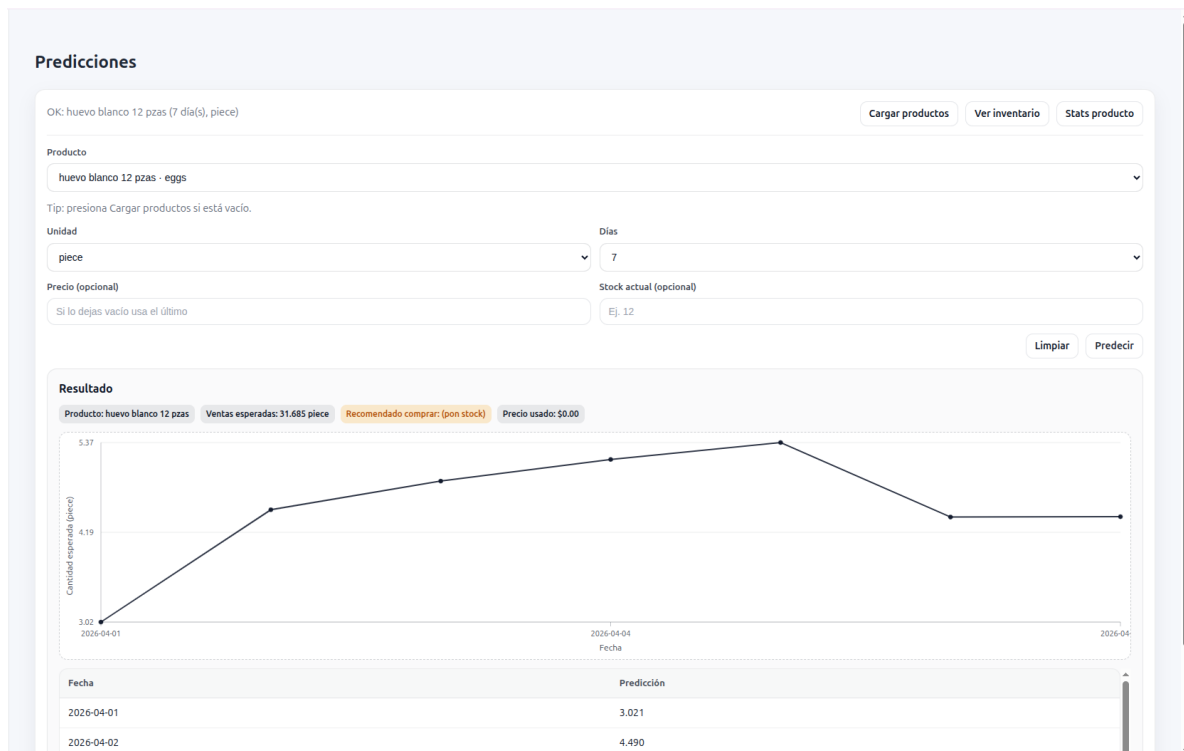


Figura 4.7. Vista del módulo de predicción de demanda.

Los resultados se presentan de forma visual representados con una gráfica y en una tabla, para que el usuario pueda utilizar esta información como apoyo para planificar compras o ajustar el inventario.

Pantalla de inventario La vista de inventario permite consultar el estado actual de existencias, apoya a manejar el stock disponible para cada producto. Esta vista es útil para identificar niveles bajos de inventario o detectar productos que requieren atención.

Inventario

Listo: 150 fila(s) de inventario. Recargar Producto Nuevo

Buscar
Nombre, categoría...

Umbral Bajo: 10 Umbral Crítico: 3

Producto	Categoría	Perecedero	Unidad	Stock	Estado	Ajuste rápido		
aceite vegetal 1 l	oil-vegetable	No	piece	4	Bajo	-1	+1	Set
acelga	fruits-vegetables	Si	piece	1	Crítico	-1	+1	Set
agua mineral 600 ml	bottled-water	No	piece	1	Crítico	-1	+1	Set
agua purificada 1.5 l	bottled-water	No	piece	1	Crítico	-1	+1	Set
aguacate	fruits-vegetables	Si	piece	0	Crítico	-1	+1	Set
aguacate criollo	avocado	Si	piece	1	Crítico	-1	+1	Set
aguacate hass	avocado	Si	piece	0	Crítico	-1	+1	Set
ajo	fruits-vegetables	Si	piece	1	Crítico	-1	+1	Set
arroz blanco 1 kg	rice-white-dry	No	piece	0	Crítico	-1	+1	Set
arroz blanco 500 g	rice-white-dry	No	piece	26	OK	-1	+1	Set
atun en aceite 140 g	canned-fish	No	piece	0	Crítico	-1	+1	Set
atun en agua 140 g	canned-fish	No	piece	0	Crítico	-1	+1	Set

Figura 4.8. Vista de consulta del estado del inventario.

Se encarga de ofrecer una idea clara del inventario actual, permitiendo agregar o disminuir el stock según sea necesario mediante botones de -1 y +1, los cuales modifican la cantidad de stock en una unidad. Además, se puede ingresar una cantidad de forma manual para realizar ajustes más específicos.

5. Análisis y discusión de resultados

En esta sección se analizan los resultados obtenidos durante el desarrollo del sistema, considerando tanto el desempeño del modelo predictivo como el funcionamiento general de la aplicación. Se destacan las ventajas del sistema desarrollado, así como su utilidad en la gestión de inventario en microempresas. Al igual que las principales limitaciones identificadas durante su implementación y se plantean posibles mejoras a futuro.

5.1. Interpretación de resultados

Los resultados obtenidos muestran que el modelo predictivo es capaz de aprender adecuadamente el comportamiento general de las ventas, alcanzando métricas dentro de rangos aceptables para su aplicación en un entorno real. Estos valores demuestran que el modelo logra identificar patrones reales en los datos, evidenciando que sus predicciones no son aleatorias, sino que están basadas en información histórica, lo que incrementa su confiabilidad para su implementación.

Sin embargo, se observaron desviaciones en puntos específicos, principalmente en picos de demanda. Este comportamiento se relaciona con la disponibilidad variable de los productos perecederos, cuya demanda puede verse afectada por factores externos como eventos, cambios en el consumo o su propia temporalidad, aspectos difíciles de predecir. Debido a estos cambios bruscos no siempre son aprendidos por el modelo.

5.2. Desempeño del sistema

La aplicación desarrollada cumple con los requerimientos funcionales planteados, permitiendo la gestión de productos, registro de ventas, control de inventario y generación de predicciones. La integración entre el modelo, el backend y el frontend se realizó de manera adecuada, permitiendo un flujo continuo de información dentro del sistema.

El uso de frameworks como FastAPI y React permitieron obtener tiempos de respuesta adecuados y una interacción fluida y sencilla con el usuario, lo que demuestra la viabilidad del sistema para ser implementado en entornos reales, particularmente en pequeños negocios.

5.3. Valor agregado del proyecto

El principal aporte de este proyecto es la integración de un sistema completo que no solo realiza predicciones, sino que también permite su despliegue directamente en un ambiente real para la gestión del inventario. A diferencia de otros trabajos similares que se enfocan únicamente en el modelo, este proyecto ofrece una solución mas robusta que combina almacenamiento de datos, visualización de información y análisis predictivo.

De este modo, el sistema fue desarrollado utilizando herramientas de código abierto, lo que facilita su implementación debido a la amplia documentación disponible y a la ausencia de costos por licencias. Esto permite su despliegue en microempresas sin necesidad de una inversión significativa ni infraestructura compleja o costosa.

5.4. Implicaciones prácticas

Los resultados obtenidos permiten apoyar la toma de decisiones en la gestión de inventario, especialmente en el reabastecimiento y el control de productos perecederos. Al igual que otros trabajos relacionados, este proyecto busca reducir pérdidas por sobreabastecimiento o desabasto

mediante el análisis de patrones de demanda. Sin embargo, a diferencia de los trabajos mencionados en la tabla 1, que están orientados a cadenas de suministro globales o procesos logísticos complejos, este sistema se enfoca en comercios locales, donde la simplicidad y la accesibilidad en su implementación son una prioridad.

El modelo proporciona estimaciones de demanda que pueden ser utilizadas directamente en la operación del negocio, sin necesidad de procesos adicionales ni de una intervención compleja por parte del usuario. Además, la interfaz del sistema facilita su uso por personas sin conocimientos técnicos, permitiendo consultar información y generar predicciones de manera sencilla. Esto mejora su compatibilidad en escenarios reales, donde los usuarios no suelen contar con conocimientos técnicos, a diferencia de otros proyectos que requieren herramientas más especializadas o configuraciones más complejas.

5.5. Limitaciones y líneas futuras

Una de las principales limitaciones del sistema es la dependencia de los datos históricos disponibles, así como la ausencia de variables externas que pueden influir en la demanda, como condiciones climáticas, promociones o factores económicos. Esto puede afectar la precisión del modelo en ciertos escenarios.

En futuras actualizaciones, se propone incorporar nuevas fuentes de información que permitan mejorar la capacidad predictiva, así como explorar modelos más avanzados que capten mejor cambios abruptos en la demanda. También se plantea la optimización del sistema y desarrollar funcionalidades adicionales, como recomendaciones automáticas de compra o alertas inteligentes para la gestión del inventario.

6. Conclusiones

El desarrollo de este proyecto permitió afrontar un problema real que es la gestión de inventarios en pequeños negocios implementando un sistema que permitió combinar un modelo predictivo de demanda con una aplicación web funcional. Los resultados obtenidos demuestran que es posible estimar el comportamiento general de las ventas con un nivel de precisión aceptable, permitiendo apoyar en la toma de decisiones en para el reabastecimiento y el control de productos perecederos y no perecederos.

Durante el desarrollo del sistema, se demostró la complejidad e importancia de trabajar con conjuntos de datos ricos en información, donde el preprocesamiento juega un papel indispensable para garantizar la calidad de los datos utilizados por el modelo. Asimismo, el ajuste adecuado del modelo representó un reto, ya que el problema presenta características particulares que influyen directamente en su desempeño, especialmente en escenarios con alta variabilidad como la demanda de productos perecederos.

Por otro lado, la construcción de la aplicación web implicó la integración de múltiples áreas de conocimiento, incluyendo el manejo de bases de datos, el diseño de arquitecturas de software, la comunicación entre sistemas mediante protocolos como HTTP, así como conceptos básicos de redes. Además, el uso de tecnologías actuales como Docker permitió facilitar el despliegue y la portabilidad del sistema, destacando la importancia de utilizar herramientas modernas dentro del desarrollo de software.

Este proyecto permitió desarrollar una solución accesible y funcional para usuarios sin conocimientos especializados, lo que incrementa la posibilidad de implementarlo en entornos reales. Se logró integrar con éxito el análisis de datos, el desarrollo de software y la experiencia de usuario en una sola herramienta.

Finalmente, este trabajo no solo demuestra la importancia de aplicar técnicas de aprendizaje automático en la gestión de inventario, sino que también plantea una base sólida para futuras mejoras, como la implementación de nuevas variables, el uso de modelos más avanzados o la implementación de funciones adicionales que permitan mejorar aún más la toma de decisiones en microempresas. La asesora se responsabiliza de guiar al alumnado y de que todos los recursos mencionados en la Factibilidad Técnica estarán disponibles para el alumnado, de modo que el proyecto de integración se pueda concluir en tiempo y forma.

Dra. Silvia Beatriz González Brambila
Asesora

Referencias

- [1] E. Chołodowicz and P. Orłowski, "Neural network control of perishable inventory with fixed shelf life products and fuzzy order refinement under time-varying uncertain demand," *Energies (Basel)*, vol. 17, no. 4, Feb. 2024.
- [2] J. S. Bravo-Aroyave, J. A. Riascos-Guerrero, E. Galván-Colonia, and J. L. Pincay-Lozada, "Estrategias basadas en inteligencia artificial para la gestión de inventarios en la cadena de suministro," *Revista Tecnología en Marcha*, vol. 37, no. 6, pp. 88–97, Jul. 2024.
- [3] I. Rey Escobar and J. E. Valle Nieto. (2025, Jul.) Transformación digital en la logística internacional: Estrategias y desafíos de la inteligencia artificial para los inventarios y cadena de suministro en las empresas exportadoras colombianas. [Online]. Available: <https://repository.upb.edu.co/handle/20.500.11912/12163>
- [4] J. Mero *et al.*, "Analysis of an inventory model in perishable products applying tabu and montecarlo simulation metaheuristic algorithm," *Ecuadorian Science Journal*, 2019.
- [5] Padilla and Ismael. (2019, Jul.) Gestión de inventario de artículos perecederos. [Online]. Available: <https://www.colibri.udelar.edu.uy/jspui/handle/20.500.12008/22859>
- [6] I. Gárate Gómez-Corisco. (2022, Jul.) Herramienta de optimización inventario en un e-commerce de bienes perecederos. [Online]. Available: <https://repositorio.comillas.edu/xmlui/handle/11531/63985>
- [7] Food and Agriculture Organization of the United Nations, "The state of food and agriculture 2019," 2019.
- [8] G. Developers, "Machine learning crash course," 2023, consultado en 2026. [Online]. Available: <https://developers.google.com/machine-learning/crash-course>
- [9] ISGlobal Barcelona Institute for Global Health, "Modelos de regresión," 2023, consultado en 2026. [Online]. Available: https://isglobal-brge.github.io/curso_R/modelos-de-regresi%C3%B3n.html
- [10] IBM, "¿qué es random forest?" 2024, consultado en 2026. [Online]. Available: <https://www.ibm.com/mx-es/think/topics/random-forest#684929713>
- [11] —, "¿qué son las redes neuronales?" 2024, consultado en marzo de 2026. [Online]. Available: <https://www.ibm.com/mx-es/think/topics/neural-networks>
- [12] Scikit-learn, "Metrics and scoring: quantifying the quality of predictions," 2024, consultado en 2026. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html
- [13] BBVA, "Arquitectura de software o sistemas: qué es y ejemplos," 2024, consultado en 2026. [Online]. Available: <https://www.bbva.com/es/innovacion/arquitectura-de-software-o-sistemas-que-es-y-ejemplos/>
- [14] ESIC Business & Marketing School, "Modelo vista controlador (mvc): qué es, cómo funciona y ventajas," 2024, consultado en 2026. [Online]. Available: <https://www.esic.edu/rethink/tecnologia/modelo-vista-controlador-que-es-como-functiona-y-ventajas-c>

- [15] IBM, “¿qué es una api?” 2024, consultado en 2026. [Online]. Available: <https://www.ibm.com/mx-es/think/topics/api>
- [16] DataCamp, “Introducción a fastapi,” 2024, consultado en 2026. [Online]. Available: <https://www.datacamp.com/es/tutorial/introduction-fastapi-tutorial>
- [17] Red Hat, “¿qué es una api rest?” 2024, consultado en 2026. [Online]. Available: <https://www.redhat.com/es/topics/api/what-is-a-rest-api>
- [18] EDteam, “¿qué es react y por qué domina el desarrollo frontend?” 2024, consultado en 2026. [Online]. Available: <https://ed.team/blog/que-es-react-y-por-que-domina-el-desarrollo-frontend>
- [19] S. I. Nikolenko, “A survey on synthetic data generation for machine learning,” *ACM Computing Surveys*, 2021.
- [20] Calendarific, “Calendarific api,” 2024, disponible en: <https://calendarific.com/api-documentation>.
- [21] Strapi, “Fastapi vs flask: Python framework comparison,” 2023, consultado en marzo de 2026. [Online]. Available: <https://strapi.io/blog/fastapi-vs-flask-python-framework-comparison>

A. Apéndices

A.1. Código fuente del clasificador de productos, clasificador_perecederos.py

```

1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  from sklearn.preprocessing import LabelEncoder
4  from sklearn.ensemble import RandomForestClassifier
5  import joblib
6  from pathlib import Path
7  from sklearn.metrics import confusion_matrix, classification_report
8  from sklearn.metrics import confusion_matrix, classification_report
9
10
11  #Apertura del dataset
12  df = pd.read_json('../Datos/datasets_originales/shelflife.json')
13
14  #Preparando el dataset para hacer la clasificacion
15  df_clasificado = df.copy()
16  columnas_usar = ['product_name', 'category_off', 'shelf_life_pantry_days',
17  ↪ 'shelf_life_fridge_days', 'shelf_life_freezer_days']
18  df_clasificado = df_clasificado[columnas_usar]
19
20  #Rellenar los valores null con ceros
21  columnas_numericas = ['shelf_life_pantry_days', 'shelf_life_fridge_days',
22  ↪ 'shelf_life_freezer_days']
23  df_clasificado[columnas_usar] = df_clasificado[columnas_usar].fillna(0)
24
25  # Clasificacion en perecedero = 1 o no_perecedero = 0
26
27  def clasificar_perecedero(fila):
28      """
29      Clasifica un producto como perecedero (1) o no perecedero (0) segun su categoria
30      ↪ y vida til.
31
32      Evala si un producto pertenece a una categoria no perecedera o si requiere
33      ↪ refrigeracin/
34      congelacin. Tambin se considera perecedero si su vida til en despensa es muy
35      ↪ corta.
36
37      Args:
38          fila (pd.Series): Fila del DataFrame con informacin del producto. Debe
39          ↪ contener las siguientes columnas:
40          - 'category_off' (str): Categoría del producto.
41          - 'shelf_life_fridge_days' (int): Das de vida til si se guarda en
42          ↪ refrigeracin.
43          - 'shelf_life_freezer_days' (int): Das de vida til si se guarda en
44          ↪ congelacin.

```



```

37         - 'shelf_life_pantry_days' (int): Das de vida til si se guarda en
           ↳ despensa.
38
39     Returns:
40         int: 1 si el producto es perecedero, 0 si no lo es.
41     """
42
43     categorias_no_perecederas = [
44         'juice-box', 'canned-fish', 'beans-dry', 'sugar-white', 'salt',
45         'soft-drinks', 'tea-bags', 'oil-vegetable', 'chips', 'rice-white-dry',
46         'pasta-dry', 'canned-tomato', 'canned-meat', 'cookies', 'cereal-box',
47         'coffee-ground', 'milk-uht', 'bottled-water', 'salsa-jarred',
48         'toothpaste', 'shampoo', 'household-detergent'
49     ]
50
51     # Si pertenece a una categoría no perecedera 0
52     if fila['category_off'] in categorias_no_perecederas:
53         return 0
54     # Si requiere de estar en un ambiente frio es 1
55     elif (fila['shelf_life_fridge_days'] > 0 or fila['shelf_life_freezer_days'] >
           ↳ 0):
56         return 1 # Perecedero (Se beneficia del frío)
57
58     #El producto no va en frío pero su vida útil en despensa es muy corta
59     elif fila['shelf_life_pantry_days'] < 20:
60         return 1 # Perecedero por vida util muy corta
61     else:
62         return 0
63
64     # Aplicar la clasificación
65     df_clasificado['perecedero'] = df.apply(clasificar_perecedero, axis=1)
66     df_clasificado.to_csv("../Datos/datasets_procesados/productos_clasificados.csv",
           ↳ ")
67
68     df_clasificado =
69     ↳ pd.read_csv("../Datos/datasets_procesados/productos_clasificados.csv")
70
71     X = df_clasificado[['category_off', 'shelf_life_pantry_days',
72     ↳ 'shelf_life_fridge_days', 'shelf_life_freezer_days']]
73     y = df_clasificado['perecedero']
74
75     # Conversion de categorias a numeros para que el modelo pueda ejecutar el
76     ↳ algoritmo
77     encoder = LabelEncoder()
78     X['category_off'] = encoder.fit_transform(X['category_off'])
79
80     # Separar los datos para entrenamiento y prueba

```

```

79 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
    ↪ random_state=42)
80
81 # Entrenar modelo
82 modelo = RandomForestClassifier(n_estimators=500, random_state=42)
83 modelo.fit(X_train, y_train)
84
85 # Evaluar modelo
86 score = modelo.score(X_test, y_test)
87 print(f"Precisión: {score:.2f}")
88
89 #Matriz de confusion
90 y_pred = modelo.predict(X_test)
91
92 #Generacion de la matriz
93 matriz_confusion = confusion_matrix(y_test, y_pred)
94 print('Matriz de confusin :')
95 print(matriz_confusion)
96
97 #Haciendo pruebas del modelo con datos nnunca vistos (completamnete nuevos)
98
99 # Los nuevos productos
100 productos_nuevos = [
101     ["Plátano tabasco 1 kg", "fruit-fresh", 5, 0, 0],
102     ["Tomate saladet 1 kg", "vegetable-fresh", 4, 0, 0],
103     ["Limón agrio 1 kg", "fruit-fresh", 7, 0, 0],
104     ["Aguacate hass 1 kg", "fruit-fresh", 3, 0, 0],
105     ["Pan dulce 1 pza", "bakery-fresh", 3, 0, 0],
106     ["Tortillas de maíz 1 kg", "tortilla", 5, 10, 30],
107     ["Jamón de pavo 250 g", "meat-processed", 0, 10, 60],
108     ["Yogur de fresa 1L", "yogurt", 0, 15, 0],
109     ["Queso panela 400 g", "cheese-soft", 0, 14, 90],
110     ["Refresco de cola 600 ml", "soda", 365, 0, 0],
111     ["Aceite vegetal 1L", "oil-vegetable", 730, 0, 0],
112     ["Café molido 250 g", "coffee-ground", 540, 0, 0],
113     ["Sal yodada 1 kg", "salt", 1825, 0, 0],
114     ["Harina de trigo 1 kg", "flour", 365, 0, 0],
115     ["Frijol negro 1 kg", "beans-dry", 365, 0, 0],
116     ["Pollo entero crudo 1 kg", "meat-raw", 0, 5, 180],
117     ["Pescado mojarra 1 kg", "fish-fresh", 0, 3, 150],
118     ["Atún enlatado 140 g", "canned-fish", 720, 0, 0],
119     ["Cereal de maíz 500 g", "cereal", 365, 0, 0],
120     ["Mantequilla 200 g", "butter", 0, 90, 180],
121 ]
122
123 df_nuevos = pd.DataFrame(productos_nuevos, columns=[
124     "product_name", "category_off", "shelf_life_pantry_days",
125     "shelf_life_fridge_days", "shelf_life_freezer_days"
126 ])

```

```
127
128 X_nuevos = df_nuevos[['category_off', 'shelf_life_pantry_days',
    ↪ 'shelf_life_fridge_days', 'shelf_life_freezer_days']]
129
130 X_nuevos['category_off'] = encoder.fit_transform(X_nuevos['category_off'])
131
132 predicciones = modelo.predict(X_nuevos)
133 df_nuevos['percedero_prediccion'] = predicciones
134
135 # Agregar las etiquetas reales
136 etiquetas_reales = [1,1,1,1,1,1,1,1,1,1,0,0,0,0,0,1,1,0,0,1]
137 df_nuevos['percedero_real'] = etiquetas_reales
138
139 # Comparar
140 df_nuevos['acierto'] = df_nuevos['percedero_prediccion'] ==
    ↪ df_nuevos['percedero_real']
141
142 # Calcular precision con productos nuevos
143 precision_nuevos = df_nuevos['acierto'].mean()
144 print(f"Precisión con productos nuevos: {precision_nuevos:.2f}")
145
146 df_nuevos_guardar = df_nuevos[['product_name', 'category_off',
    ↪ 'shelf_life_pantry_days', 'shelf_life_fridge_days', 'shelf_life_freezer_days',
    ↪ 'percedero_real']]
147 ultimo_indice =
    ↪ pd.read_csv('../Datos/datasets_procesados/productos_clasificados.csv',
    ↪ usecols=['Unnamed: 0']).iloc[-1, 0]
148 df_nuevos_guardar.index = range(ultimo_indice + 1, ultimo_indice + 1 +
    ↪ len(df_nuevos))
149 df_nuevos_guardar.to_csv('../Datos/datasets_procesados/productos_clasificados.'
    ↪ csv', mode='a', header=False, index=True)
150
151 df_nuevos.head(20)
152
153
154 #En esta seccion se guardan los modelos ya entrenados
155 # Directorio de este script
156 BASE_DIR = Path.cwd()
157
158 # Subir un nivel y entrar a Modelos/
159 MODELS_DIR = BASE_DIR.parent / ".." / "Modelos"
160
161 # Crear carpeta si no existe
162 MODELS_DIR.mkdir(parents=True, exist_ok=True)
163
164 # Guardar modelo y encoder
165 joblib.dump(modelo, MODELS_DIR / "percedero_rf.pkl")
166 joblib.dump(encoder, MODELS_DIR / "category_encoder.pkl")
```

A.2. Código fuente del módulo obtener_feriados_CDMX.py

```
1      import requests
2      import pandas as pd
3      from pathlib import Path
4      import datetime
5
6      def obtener_feriados_calendarific(
7          api_key: str,
8          country: str = "MX",
9          year: int = None,
10         out_file: str = "../../Datos/datasets_originales/eventos_cdmx.csv",
11         base_url: str = "https://calendarific.com/api/v2/holidays"
12     ):
13
14         """
15         Descarga los días feriados de Calendarific y los guarda en un CSV con
16         ↪ formato:
17         date,event,scope
18
19         Parámetros:
20             api_key (str): API Key de Calendarific.
21             country (str): Código de país (por defecto 'MX').
22             year (int): Año a consultar (por defecto año actual).
23             out_file (str): Ruta donde guardar el CSV.
24             base_url (str): URL base de la API (no modificar normalmente).
25
26         Retorna:
27             pd.DataFrame con los feriados obtenidos.
28         """
29
30         if year is None:
31             year = datetime.datetime.now().year
32
33         out_path = Path(out_file)
34         out_path.parent.mkdir(parents=True, exist_ok=True)
35
36         # Llamada a la API Calendarific
37         params = {
38             "api_key": api_key,
39             "country": country,
40             "year": year,
41         }
42
43         response = requests.get(base_url, params=params)
44
45         if response.status_code != 200:
46             raise RuntimeError(f"Error en API ({response.status_code}):
47             ↪ {response.text}")
```

```

46
47     data = response.json()
48
49     if "response" not in data or "holidays" not in data["response"]:
50         raise ValueError(f"Formato inesperado de respuesta: {data}")
51
52     holidays = data["response"]["holidays"]
53
54     # Procesamiento de los Datos
55
56     rows = []
57     for holiday in holidays:
58         name = holiday.get("name", "")
59         date_iso = holiday.get("date", {}).get("iso", "")
60         types = holiday.get("type", [])
61         type_str = ", ".join(types) if isinstance(types, list) else str(types)
62
63         rows.append({
64             "date": date_iso[:10], # yyyy-mm-dd
65             "event": name,
66             "scope": type_str or "holiday"
67         })
68
69     df = pd.DataFrame(rows)
70
71     # Guardar en formato csv
72
73     df.to_csv(out_path, index=False, encoding="utf-8")
74     print(f"Se guardaron {len(df)} feriados en: {out_path}")
75     print(df.head())
76
77     return df
78
79     obtener_feridos_calendarific(
80         api_key="bvEJzyh0h4J3czQcdJXHzasOSLbL9Bsr",
81         year=datetime.datetime.now().year
82     )

```

A.3. Código fuente del módulo `normalizacion_fruitsvegetables_temporadas.py`

```

1     import pandas as pd
2     import numpy as np
3
4     df_temp = pd.read_csv("../Datos/datasets_originales/frutas_verduras_tempor_
5         ↪ ada.csv")
6     df_temp_cp = df_temp.copy()

```

```

7      def cambiar_valores_columna(df : pd.DataFrame, nombre_columna: str, valor:
      ↪ any) -> pd.DataFrame:
8          """
9          Reemplaza todos los valores de una columna especfica en un DataFrame por
      ↪ un nuevo valor.
10
11          Parametros
12          -----
13          df : pandas.DataFrame
14              El DataFrame que contiene la columna a modificar.
15          nombre_columna : str
16              El nombre de la columna cuyos valores seran reemplazados.
17          valor : any
18              El nuevo valor que se asignara a todos los elementos de la columna.
19              Puede ser un valor escalar (str, int, float, bool, etc.).
20
21          Retorna
22          -----
23          pandas.DataFrame
24              Una copia del DataFrame con la columna modificada.
25              No altera el DataFrame original.
26          """
27
28          if nombre_columna not in df.columns:
29              raise KeyError(f"La columna '{nombre_columna}' no existe en el
      ↪ DataFrame.")
30          df_copy = df.copy()
31          df_copy[nombre_columna] = valor
32          return df_copy
33
34
35          #Se cambia solo se cambia la categoria a 'fruits-vegetables'
36          df_temp_cp = cambiar_valores_columna(df_temp_cp, 'tipo', 'fruits-vegetables')
37          df_temp_cp.tail(3)
38
39      def extraer_temporadas(df, col="temporada_pico"):
40          """
41          Crea tres nuevas columnas en el DataFrame con informacin sobre la
      ↪ temporada.
42
43          - todo_el_anio: True si en el texto aparece "todo el año"
44          - temporada_pico_inicio: nmero del mes donde inicia la temporada (1 =
      ↪ enero)
45          - temporada_pico_fin: nmero del mes donde termina la temporada (12 =
      ↪ diciembre)
46          """
47
48          # Diccionario para convertir abreviaturas de meses a numeros
49          meses = {

```

```
50         "ene": 1, "feb": 2, "mar": 3, "abr": 4, "may": 5, "jun": 6,
51         "jul": 7, "ago": 8, "sep": 9, "oct": 10, "nov": 11, "dic": 12
52     }
53
54     # Lista donde guardar los resultados
55     todo_el_anio = []
56     mes_inicio = []
57     mes_fin = []
58
59     # Recorrido de cada valor de la columna
60     for texto in df[col]:
61         # Si el valor está vacío o es NaN
62         if pd.isna(texto):
63             todo_el_anio.append(False)
64             mes_inicio.append(np.nan)
65             mes_fin.append(np.nan)
66             continue
67
68         texto = texto.lower().replace("-", "-") # reemplaza guiones raros
69         es_todo_anio = "todo el año" in texto or "Todo el año" in texto
70
71         # Buscar un rango con guion
72         if "-" in texto:
73             # Tomamos las primeras tres letras de cada mes
74             partes = texto.split("-")
75             mes_i = partes[0].strip()[:3]
76             mes_f = partes[1].strip()[:3]
77             inicio = meses.get(mes_i, np.nan)
78             fin = meses.get(mes_f, np.nan)
79         else:
80             # En caso de no haber guion
81             inicio = fin = np.nan
82             for m in meses:
83                 if m in texto:
84                     inicio = fin = meses[m]
85                     break
86
87         todo_el_anio.append(es_todo_anio)
88         mes_inicio.append(inicio)
89         mes_fin.append(fin)
90
91     # Agregar las tres nuevas columnas al DataFrame
92     df["todo_el_anio"] = todo_el_anio
93     df["temporada_pico_inicio"] = mes_inicio
94     df["temporada_pico_fin"] = mes_fin
95
96     return df
97
98
```

```
99     df = extraer_temporadas(df_temp_cp)
100     df.to_csv("../Datos/datasets_procesados/frutas-verduras_temporada_normaliz_
    ↪     ado.csv")
```

A.4. Código fuente del módulo generar_eventos_productos.py

```
1     import pandas as pd
2     from pathlib import Path
3     import unicodedata
4
5     def normalize_text(texto):
6         """
7         Deja el texto en minúsculas, sin acentos ni caracteres raros,
8         y sin espacios duplicados. Sirve para poder comparar nombres.
9         """
10        if pd.isna(texto):
11            return ""
12
13        # Pasar a minúsculas y quitar espacios al inicio y final
14        texto = texto.lower().strip()
15
16        # Quitar acentos
17        texto = unicodedata.normalize("NFD", texto)
18        texto = "".join(c for c in texto if unicodedata.category(c) != "Mn")
19
20        # Reemplazar algunos signos por espacio
21        for rep in [",", ".", "(", ")", "-", "_"]:
22            texto = texto.replace(rep, " ")
23
24        # Quitar espacios duplicados
25        texto = " ".join(texto.split())
26
27        return texto
28
29    def month_in_range(mes, inicio, fin, todo_el_anio):
30        """
31        Revisa si el mes cae dentro de la temporada de un producto.
32        """
33        if todo_el_anio:
34            return True
35
36        if pd.isna(inicio) or pd.isna(fin):
37            return False
38
39        inicio = int(inicio)
40        fin = int(fin)
41
42        # Temporada normal, por ejemplo de mayo (5) a agosto (8)
```



```

43     if inicio <= fin:
44         return inicio <= mes <= fin
45     else:
46         # Temporada que cruza el ao, por ejemplo de diciembre (12) a marzo (3)
47         return mes >= inicio or mes <= fin
48
49 def generar_eventos_productos(data_dir="../../Datos",
↳ out_file="eventos_productos.csv"):
50     """
51     Genera el archivo eventos_productos.csv a partir de:
52         - eventos_cdmx.csv
53         - frutas-verduras_temporada_normalizado.csv
54         - productos_clasificados.csv
55
56     Parámetros:
57         data_dir: carpeta donde están los CSV (por defecto '../datasets')
58         out_file: nombre del archivo de salida dentro de esa carpeta
59
60     Regresa:
61         df_eventos_prod: DataFrame con las relaciones evento-producto
62         con columnas: [event, date, producto_relacionado]
63     """
64     # Convertir rutas a objetos Path
65     DATA_DIR = Path(data_dir)
66     OUT_PATH = DATA_DIR / "eventos" / out_file
67
68     # Cargar archivos
69     df_eventos = pd.read_csv(DATA_DIR / "datasets_originales"
↳ / "eventos_cdmx.csv", parse_dates=["date"])
70
71     df_temp = pd.read_csv(DATA_DIR / "datasets_procesados"
↳ / "frutas-verduras_temporada_normalizado.csv")
72
73     df_prod = pd.read_csv(DATA_DIR / "datasets_procesados"
↳ / "productos_clasificados.csv")
74
75     # Crear columnas "normalizadas" para poder comparar texto
76     df_temp["key_temp"] = df_temp["nombre"].apply(normalize_text)
77     df_prod["key_prod"] = df_prod["product_name"].apply(normalize_text)
78
79     mapa_nombre_producto = {}
80
81
82     for i, row_temp in df_temp.iterrows():
83         key_temp = row_temp["key_temp"]
84
85         # Buscar productos cuyo nombre normalizado contenga el nombre de la
↳ fruta/verdura

```

```

86     candidatos = df_prod[df_prod["key_prod"].str.contains(key_temp,
87         ↪ na=False)]
88
89     if len(candidatos) == 0:
90         # Si no encuentra un producto equivalente, usa el nombre de la
91         ↪ fruta/verdura
92         mapa_nombre_producto[key_temp] = row_temp["nombre"]
93     else:
94         producto = candidatos.iloc[0]["product_name"]
95         mapa_nombre_producto[key_temp] = producto
96
97     # Lista donde se guardan todas las filas del CSV final
98     rows = []
99
100    print("Generando relaciones evento -> frutas/verduras en temporada...")
101
102    for i, ev in df_eventos.iterrows():
103        event_name = ev["event"]
104        fecha_evento = ev["date"]
105        mes_evento = fecha_evento.month
106
107        # Para cada fruta/verdura revisa si está en temporada ese mes
108        for j, row_temp in df_temp.iterrows():
109            todo_el_anio = bool(row_temp.get("todo_el_anio", False))
110
111            en_temp = month_in_range(
112                mes_evento,
113                row_temp["temporada_pico_inicio"],
114                row_temp["temporada_pico_fin"],
115                todo_el_anio
116            )
117
118            if en_temp:
119                key_temp = row_temp["key_temp"]
120                producto_relacionado = mapa_nombre_producto.get(key_temp,
121                    ↪ row_temp["nombre"])
122
123                rows.append({
124                    "event": event_name,
125                    "date": fecha_evento,
126                    "producto_relacionado": producto_relacionado
127                })
128
129        # Si el nombre del evento contiene cierto texto, se relacionan algunas
130        ↪ categorias
131    EVENT_CATEGORY_RULES = {
132        "christmas": [
133            "disposable-cups", "disposable-plates", "disposable-cutlery",
134            "disposable-trays", "disposable-containers", "disposable-lids",

```

```

131         "beer", "whisky", "rum", "vodka", "brandy", "soda",
           ↳ "bottled-water"
132     ],
133     "new year's eve": [
134         "disposable-cups", "disposable-plates", "disposable-cutlery",
135         "beer", "whisky", "rum", "vodka", "brandy", "soda",
           ↳ "bottled-water"
136     ],
137     "day of the holy kings": [
138         "disposable-cups", "disposable-plates",
139         "soda", "bottled-water", "canned-milk"
140     ],
141     "children's day": [
142         "soda", "bottled-water", "snacks"
143     ],
144     "fathers' day": [
145         "beer", "whisky", "rum", "vodka", "brandy", "soda"
146     ],
147     "mothers' day": [
148         "soda", "bottled-water"
149     ],
150     "independence day": [
151         "disposable-plates", "disposable-cups", "disposable-cutlery",
           ↳ "beer", "soda"
152     ],
153 }
154
155 # Palabras clave para buscar también por texto en el nombre del producto
156 EXTRA_KEYWORDS = {
157     "christmas": ["vasos desechables", "plato unicel", "platos
           ↳ desechables"],
158     "new year's eve": ["vasos desechables", "plato unicel", "platos
           ↳ desechables"],
159 }
160
161 for i, ev in df_eventos.iterrows():
162     event_name = ev["event"]
163     fecha_evento = ev["date"]
164     event_lower = event_name.lower()
165
166 # Reglas por categoría (usa la columna category_off)
167 for pattern, categories in EVENT_CATEGORY_RULES.items():
168     if pattern in event_lower:
169         for categoria in categories:
170             # Tomar todos los productos que pertenezcan a esa categoria
171             productos_categoria = df_prod[df_prod["category_off"] ==
           ↳ categoria]
172
173             for k, prod in productos_categoria.iterrows():

```

```

174         rows.append({
175             "event": event_name,
176             "date": fecha_evento,
177             "producto_relacionado": prod["product_name"]
178         })
179
180     # Reglas por palabras clave en el nombre del producto
181     for pattern, keywords in EXTRA_KEYWORDS.items():
182         if pattern in event_lower:
183             for kw in keywords:
184                 kw_norm = normalize_text(kw)
185                 # Buscar productos cuyo nombre normalizado contenga esa
186                 ↪ palabra clave
187                 productos_kw =
188                 ↪ df_prod[df_prod["key_prod"].str.contains(kw_norm,
189                 ↪ na=False)]
190
191             for k, prod in productos_kw.iterrows():
192                 rows.append({
193                     "event": event_name,
194                     "date": fecha_evento,
195                     "producto_relacionado": prod["product_name"]
196                 })
197
198     df_eventos_prod = pd.DataFrame(rows)
199
200     # Eliminar duplicados por si se repiten combinaciones event/producto
201     df_eventos_prod = df_eventos_prod.drop_duplicates().reset_index(drop=True)
202
203     OUT_PATH.parent.mkdir(parents=True, exist_ok=True)
204     df_eventos_prod.to_csv(OUT_PATH, index=False, encoding="utf-8")
205
206     return df_eventos_prod
207
208 df_eventos_prod = generar_eventos_productos()
209
210 print(df_eventos_prod.head())

```

A.5. Código fuente del módulo completar_dataSet_ventas.py

```

1     import pandas as pd
2     from pathlib import Path
3     import unicodedata
4
5     #Apertura de los datasets en un DataFrame
6     DATA_DIR = Path("../..../Datos")
7
8     df_ventas = pd.read_csv(DATA_DIR / "datasets_originales" / 'ventas.csv')

```

```

9      df_clasificados = pd.read_csv(DATA_DIR / "datasets_procesados"
    ↪      /'productos_clasificados.csv')
10
11      #Del DataFrame donde estan los productos clasificados solo se usaran las
    ↪      siguientes dos columnas
12      df_clasificados = df_clasificados[['product_name', 'perecedero']]
13      df_clasificados.head(3)
14
15      #Se hace una copia del DataFrame de las ventas
16      df_ventas_cp = df_ventas.copy()
17
18      #Se Hace la union del DataFrame de perecederos, es decir a cada producto se le
    ↪      asigna si es perecedero o no
19      df_ventas_perecederos = pd.merge(df_ventas_cp, df_clasificados, how="left",
    ↪      on= "product_name")
20      df_ventas_perecederos.loc[df_ventas_perecederos['category_off'] ==
    ↪      'fruits-vegetables', 'perecedero'] = 1
21      df_ventas_perecederos[df_ventas_perecederos['perecedero'].isna()]
22
23      #Se crean las siguientes tres columnas con valores por defecto
24      df_ventas_perecederos['en_temporada'] = 0
25      df_ventas_perecederos['temp_inicio_mes'] = 1
26      df_ventas_perecederos['temp_fin_mes'] = 12
27
28      #Se abre el DataFrame con las temporadas de todas las frutas y verduras
29      df_frutas_verduras_temp = pd.read_csv(DATA_DIR / "datasets_procesados"
    ↪      /'frutas-verduras_temporada_normalizado.csv')
30
31      #Funcion que se encarga de acer una normalizacion simple de los nombres de los
    ↪      productos
32      def normalizar(texto):
33          """
34          La funcion realiza lo siguiente:
35          - Convierte el texto a minúsculas.
36          - Elimina espacios en blanco al inicio y al final.
37          - Quita acentos y caracteres diacríticos (normalización Unicode NFD).
38          - Convierte valores NaN en una cadena vacía.
39
40          Parámetros
41          -----
42          texto : str o cualquier tipo convertible a string
43                  Texto original a normalizar.
44          """
45
46          if pd.isna(texto):
47              return ""
48          texto = str(texto).lower().strip()
49          texto = unicodedata.normalize("NFD", texto)

```

```

50         texto = "".join(c for c in texto if unicodedata.category(c) != "Mn") #
           ↳ quita acentos
51         return texto
52
53     # Crear columnas normalizadas
54     df_frutas_verduras_temp['nombre'] =
           ↳ df_frutas_verduras_temp['nombre'].apply(normalizar)
55     df_ventas_perecederos['product_name'] =
           ↳ df_ventas_perecederos['product_name'].apply(normalizar)
56
57     #De todas las frutas y verduras las busca en el dataFrame de ventas para poder
           ↳ asignar el inicio y final de la temporada
58     for _, row in df_frutas_verduras_temp.iterrows():
59         nombre = row['nombre']
60
61         # Buscar los product_name que contengan ese nombre
62         mask = df_ventas_perecederos['product_name'].str.contains(nombre,
           ↳ na=False)
63
64         # Asignar inicio y fin de temporada
65         df_ventas_perecederos.loc[mask, 'temp_inicio_mes'] =
           ↳ row['temporada_pico_inicio']
66         df_ventas_perecederos.loc[mask, 'temp_fin_mes'] =
           ↳ row['temporada_pico_fin']
67
68     #Calcula si un producto est en temporada segn el mes de la venta y los meses
           ↳ de inicio y fin de su temporada.
69     df_ventas_perecederos['fecha'] =
           ↳ pd.to_datetime(df_ventas_perecederos['fecha'])
70     mes = df_ventas_perecederos['fecha'].dt.month
71     ini = df_ventas_perecederos['temp_inicio_mes']
72     fin = df_ventas_perecederos['temp_fin_mes']
73
74     #temporada normal (inicio <= fin)
75     caso_1 = (ini <= fin) & (mes >= ini) & (mes <= fin)
76
77     #temporada que cruza año (inicio > fin)
78     caso_2 = (ini > fin) & ((mes >= ini) | (mes <= fin))
79
80     df_ventas_perecederos['en_temporada'] = 0
81     df_ventas_perecederos.loc[caso_1 | caso_2, 'en_temporada'] = 1
82
83
84     '''
85     Debido a que hay productos disponibles todo el año (del mes 1 al 12),
86     la temporada de inicio y la temporada de fin aparecen con valores nulos.
87     Para garantizar que estos productos sean considerados como disponibles
88     durante cualquier mes, se asignan valores por defecto:
89     - temp_inicio_mes = 1 (enero)

```

```

90     - temp_fin_mes = 12 (diciembre)
91     '''
92     df_ventas_perecederos['temp_inicio_mes'] =
93     ↪ df_ventas_perecederos['temp_inicio_mes'].fillna(1)
94     df_ventas_perecederos['temp_fin_mes'] =
95     ↪ df_ventas_perecederos['temp_fin_mes'].fillna(12)
96
97     #Calcula si un producto est en temporada segn el mes de la venta y los meses
98     ↪ de inicio y fin de su temporada.
99
100     df_ventas_perecederos['fecha'] =
101     ↪ pd.to_datetime(df_ventas_perecederos['fecha'])
102     mes = df_ventas_perecederos['fecha'].dt.month
103     ini = df_ventas_perecederos['temp_inicio_mes']
104     fin = df_ventas_perecederos['temp_fin_mes']
105
106     #temporada normal (inicio <= fin)
107     caso_1 = (ini <= fin) & (mes >= ini) & (mes <= fin)
108
109     #temporada que cruza año (inicio > fin)
110     caso_2 = (ini > fin) & ((mes >= ini) | (mes <= fin))
111
112     df_ventas_perecederos['en_temporada'] = 0
113     df_ventas_perecederos.loc[caso_1 | caso_2, 'en_temporada'] = 1
114
115     df_ventas_perecederos.to_csv(DATA_DIR/ "datasets_procesados"
116     ↪ /'ventas_normalizado.csv')

```

A.6. Código fuente del módulo entrenamiento_modelo_demanda.py

```

1     import pandas as pd
2     import numpy as np
3     import matplotlib.pyplot as plt
4     from pathlib import Path
5     import unicodedata
6     from sklearn.preprocessing import StandardScaler
7     import tensorflow as tf
8     from tensorflow import keras
9     from tensorflow.keras import layers
10    from datetime import timedelta, datetime
11    import joblib
12    from pathlib import Path
13    from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
14
15
16    # Ruta base de los datasets
17    DATA_DIR = Path("../..../Datos")

```

```

18
19     # Cargar ventas y eventos
20     df_ventas = pd.read_csv(DATA_DIR / "datasets_procesados"
21                             ↪ / "ventas_normalizado.csv")
22
23     df_eventos = pd.read_csv(DATA_DIR / "eventos" / "eventos_productos.csv")
24
25     def normalize_text(texto: str) -> str:
26         """
27         Deja el texto en minúsculas, sin acentos y sin espacios duplicados.
28         """
29         if pd.isna(texto):
30             return ""
31         texto = str(texto).lower().strip()
32         texto = unicodedata.normalize("NFD", texto)
33         texto = "".join(c for c in texto if unicodedata.category(c) != "Mn")
34         texto = " ".join(texto.split())
35         return texto
36
37     # Asegurar tipos de fecha
38     df_ventas["fecha"] = pd.to_datetime(df_ventas["fecha"])
39     df_eventos["date"] = pd.to_datetime(df_eventos["date"])
40
41     # Claves normalizadas de producto
42     df_ventas["product_key"] = df_ventas["product_name"].apply(normalize_text)
43     df_eventos["product_key"] =
44     ↪ df_eventos["producto_relacionado"].apply(normalize_text)
45
46     # Merge left para marcar eventos por (fecha, producto)
47     df_eventos_reduc = df_eventos[["event", "date",
48     ↪ "product_key"]].drop_duplicates()
49
50     df_ventas_evt = df_ventas.merge(
51         df_eventos_reduc,
52         left_on=["fecha", "product_key"],
53         right_on=["date", "product_key"],
54         how="left"
55     )
56
57     # hay_evento = 1 si existe un evento para ese producto en esa fecha
58     df_ventas_evt["hay_evento"] = df_ventas_evt["event"].notna().astype(int)
59
60     # Ya no se necesita la columna date del merge
61     df_ventas_evt = df_ventas_evt.drop(columns=["date"])
62
63     ventas = df_ventas_evt.copy()
64
65     # Calendario
66     ventas["anio"] = ventas["fecha"].dt.year
67     ventas["mes"] = ventas["fecha"].dt.month

```



```

64     ventas["dia"] = ventas["fecha"].dt.day
65     ventas["dia_semana"] = ventas["fecha"].dt.weekday
66     ventas["es_fin_semana"] = ventas["dia_semana"].isin([5, 6]).astype(int)
67
68     # Codificación ciclica de mes y día de semana
69     ventas["mes_sin"] = np.sin(2 * np.pi * ventas["mes"] / 12)
70     ventas["mes_cos"] = np.cos(2 * np.pi * ventas["mes"] / 12)
71     ventas["dow_sin"] = np.sin(2 * np.pi * ventas["dia_semana"] / 7)
72     ventas["dow_cos"] = np.cos(2 * np.pi * ventas["dia_semana"] / 7)
73
74     # IDs de producto y categoría
75     ventas["product_id"], product_uniques = pd.factorize(ventas["product_name"])
76     ventas["category_id"], category_uniques = pd.factorize(ventas["category_off"])
77
78     n_products = len(product_uniques)
79     n_categories = len(category_uniques)
80     print("n_products:", n_products, "n_categories:", n_categories)
81
82     ventas = ventas.sort_values(["product_id", "fecha"])
83
84     ventas = ventas.sort_values(["product_id", "fecha"])
85
86     def crear_ventanas(df: pd.DataFrame) -> pd.DataFrame:
87         """
88         Crea variables con información de ventas pasadas para cada producto.
89
90         La función agrega:
91         - Ventas de días anteriores (1, 7 y 14 días atrás).
92         - Promedios de ventas recientes (7, 28 y 90 días),
93           usando solo datos del pasado para no afectar el entrenamiento.
94
95         Estas variables ayudan al modelo a aprender patrones
96         como tendencias y comportamientos semanales o mensuales.
97
98         Parámetros:
99         df : DataFrame con columnas `product_id` y `ventas`.
100
101         Retorna:
102         El mismo DataFrame con nuevas columnas de apoyo para predicción.
103         """
104         df = df.copy()
105         g = df.groupby("product_id")["ventas"]
106         df["lag_1"] = g.shift(1)
107         df["lag_7"] = g.shift(7)
108         df["lag_14"] = g.shift(14)
109         df["media_7"] = g.shift(1).rolling(7).mean()
110         df["media_28"] = g.shift(1).rolling(28).mean()
111         df["media_90"] = g.shift(1).rolling(90).mean()
112         return df

```

```

113
114     ventas = crear_ventanas(ventas)
115
116     # Eliminar filas sin historial suficiente
117     ventas_modelo = ventas.dropna(subset=[
118         "lag_1", "lag_7", "lag_14",
119         "media_7", "media_28", "media_90"
120     ]).copy()
121
122     fecha_corte_test = pd.to_datetime("2025-06-01")
123     VAL_DAYS = 60
124     fecha_inicio_val = fecha_corte_test - pd.Timedelta(days=VAL_DAYS)
125
126     mask_train = ventas_modelo["fecha"] < fecha_inicio_val
127     mask_val = (ventas_modelo["fecha"] >= fecha_inicio_val) &
128     ↪ (ventas_modelo["fecha"] < fecha_corte_test)
129     mask_test = ventas_modelo["fecha"] >= fecha_corte_test
130
131     df_train = ventas_modelo.loc[mask_train].copy()
132     df_val = ventas_modelo.loc[mask_val].copy()
133     df_test = ventas_modelo.loc[mask_test].copy()
134
135     features = [
136         "product_id", "category_id", "perecedero",
137         "precio", "en_temporada", "hay_evento",
138         "anio", "mes", "dia_semana", "es_fin_semana",
139         "mes_sin", "mes_cos", "dow_sin", "dow_cos",
140         "lag_1", "lag_7", "lag_14", "media_7", "media_28", "media_90"
141     ]
142     target_col = "ventas"
143
144     # Asegurar numérico básico
145     for c in features:
146         df_train[c] = pd.to_numeric(df_train[c], errors="coerce")
147         df_val[c] = pd.to_numeric(df_val[c], errors="coerce")
148         df_test[c] = pd.to_numeric(df_test[c], errors="coerce")
149
150     df_train[target_col] = pd.to_numeric(df_train[target_col], errors="coerce")
151     df_val[target_col] = pd.to_numeric(df_val[target_col], errors="coerce")
152     df_test[target_col] = pd.to_numeric(df_test[target_col], errors="coerce")
153
154     # Quitar filas con NaN en lo necesario
155     df_train = df_train.dropna(subset=features + [target_col])
156     df_val = df_val.dropna(subset=features + [target_col])
157     df_test = df_test.dropna(subset=features + [target_col])
158
159     X_train = df_train[features].astype("float32").to_numpy()
160     X_val = df_val[features].astype("float32").to_numpy()
161     X_test = df_test[features].astype("float32").to_numpy()

```

```

161     y_train = df_train[target_col].astype("float32").to_numpy()
162     y_val    = df_val[target_col].astype("float32").to_numpy()
163     y_test   = df_test[target_col].astype("float32").to_numpy()
164
165
166     # Escalar tod menos los id's (product_id y category_id)
167     id_idx = [features.index("product_id"), features.index("category_id")]
168     num_idx = [i for i in range(len(features)) if i not in id_idx]
169
170     scaler = StandardScaler()
171     X_train[:, num_idx] = scaler.fit_transform(X_train[:, num_idx])
172     X_val[:, num_idx]   = scaler.transform(X_val[:, num_idx])
173     X_test[:, num_idx]  = scaler.transform(X_test[:, num_idx])
174
175     # Target en log
176     y_train_t = np.log1p(y_train)
177     y_val_t    = np.log1p(y_val)
178     y_test_t   = np.log1p(y_test)
179
180     input_dim = X_train.shape[1]
181
182     model = keras.Sequential([
183         layers.Input(shape=(input_dim,)),
184         layers.Dense(256, activation="relu"),
185         layers.Dropout(0.25),
186         layers.Dense(128, activation="relu"),
187         layers.Dropout(0.25),
188         layers.Dense(64, activation="relu"),
189         layers.Dense(1)
190     ])
191
192     model.compile(
193         optimizer=keras.optimizers.Adam(learning_rate=3e-4),
194         loss=keras.losses.Huber(delta=0.5),
195         metrics=["mae"]
196     )
197
198     model.summary()
199
200     callbacks = [
201         keras.callbacks.EarlyStopping(monitor="val_loss", patience=8,
202             ↪ restore_best_weights=True),
203         keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
204             ↪ patience=3, min_lr=1e-6, verbose=1),
205     ]
206
207     history = model.fit(
208         X_train, y_train_t,
209         validation_data=(X_val, y_val_t),

```

```

208         epochs=200,
209         batch_size=256,
210         callbacks=callbacks,
211         verbose=1
212     )
213
214     # 1. Predicción y transformación inversa
215     y_pred_log = model.predict(X_test, verbose=0).ravel()
216     y_pred = np.expml(y_pred_log)
217
218     # Evitar predicciones negativas
219     y_pred = np.clip(y_pred, 0, None)
220
221     # 2. Métricas Base (Unidades absolutas)
222     mae = mean_absolute_error(y_test, y_pred)
223     rmse = np.sqrt(mean_squared_error(y_test, y_pred))
224
225     # 3. WAPE (Weighted Absolute Percentage Error)
226     sumaErrores = np.sum(np.abs(y_test - y_pred))
227     sumaVentasReales = np.sum(y_test)
228     wape = sumaErrores / sumaVentasReales if sumaVentasReales != 0 else 0
229
230     # 4. sMAPE (Symmetric Mean Absolute Percentage Error)
231     denominador_smape = (np.abs(y_test) + np.abs(y_pred)) / 2.0
232     # Mscara para evitar divisin por cero cuando tanto prediccin como real son 0
233     smape_mask = denominador_smape != 0
234     smape = np.mean(np.abs(y_test[smape_mask] - y_pred[smape_mask]) /
235         ↪ denominador_smape[smape_mask])
236
237     # 5. R2 Score (Coeficiente de determinación)
238     r2 = r2_score(y_test, y_pred)
239
240     print(f"MAE (Error Promedio)           : {mae:.4f} unidades")
241     print(f"RMSE (Penalización atípicos)    : {rmse:.4f} unidades")
242     print(f"WAPE (Error Porcentual Global): {wape * 100:.2f}%")
243     print(f"sMAPE (Error Simétrico)         : {smape * 100:.2f}%")
244     print(f"R2 Score                        : {r2:.4f}")
245
246     # Carpeta destino de modelos
247     OUT_DIR = Path("../")
248     OUT_DIR.mkdir(parents=True, exist_ok=True)
249
250     # 1) Guardar modelo Keras (recomendado .keras)
251     model_path = OUT_DIR / "nn_sales_model.keras"
252     model.save(model_path)
253
254     # 2) Guardar scaler
255     scaler_path = OUT_DIR / "scaler.joblib"
256     joblib.dump(scaler, scaler_path)

```

```

256
257     # 3) Guardar metadata necesaria para inferencia
258     meta = {
259         "features": features,
260         "target_col": target_col,
261         "id_idx": id_idx,
262         "num_idx": num_idx,
263         "product_uniques": list(product_uniques),
264         "category_uniques": list(category_uniques),
265     }
266
267     meta_path = OUT_DIR / "meta.joblib"
268     joblib.dump(meta, meta_path)

```

A.7. Lenguaje de Definición de Datos scriptBD.mysql

```

1     CREATE DATABASE IF NOT EXISTS pt;
2     USE pt;
3
4     -- drop database pt;
5
6     CREATE TABLE IF NOT EXISTS products (
7         id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
8         product_name VARCHAR(255) NOT NULL,
9         category_off VARCHAR(120),
10        shelf_life_pantry_days INT,
11        shelf_life_fridge_days INT,
12        shelf_life_freezer_days INT,
13        perecedero BOOLEAN NOT NULL DEFAULT 0,
14
15        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
16    );
17
18    CREATE TABLE IF NOT EXISTS sales (
19        id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
20        sold_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
21        total DECIMAL(12,2) NOT NULL DEFAULT 0,
22        payment_method VARCHAR(40) NULL,
23        notes VARCHAR(255) NULL
24    );
25
26    CREATE TABLE IF NOT EXISTS sale_items (
27        id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
28        sale_id BIGINT UNSIGNED NOT NULL,
29        product_id BIGINT UNSIGNED NOT NULL,
30
31        qty DECIMAL(12,3) NOT NULL,
32

```

```

33     unit ENUM('kg','g','piece','pack','box','lt','ml') NOT NULL DEFAULT 'piece',
34     unit_price DECIMAL(12,2) NOT NULL,
35     subtotal DECIMAL(12,2) NOT NULL,
36
37     CONSTRAINT fk_sale_items_sale FOREIGN KEY (sale_id) REFERENCES sales(id) ON
38     → DELETE CASCADE,
39     CONSTRAINT fk_sale_items_product FOREIGN KEY (product_id) REFERENCES
40     → products(id),
41     INDEX idx_sale_items_sale (sale_id),
42     INDEX idx_sale_items_product (product_id)
43 );
44
45 CREATE TABLE IF NOT EXISTS inventory_movements (
46     id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
47     product_id BIGINT UNSIGNED NOT NULL,
48     type ENUM('IN','OUT','ADJUST') NOT NULL,
49
50     qty DECIMAL(12,3) NOT NULL,
51
52     -- Unidad en la que se capturó el movimiento
53     unit ENUM('kg','g','piece','pack','box','lt','ml') NOT NULL DEFAULT 'piece',
54
55     unit_cost DECIMAL(12,2) NULL,
56     reason VARCHAR(255) NULL,
57     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
58
59     CONSTRAINT fk_inv_mov_product FOREIGN KEY (product_id) REFERENCES products(id)
60     → ON DELETE CASCADE,
61     INDEX idx_inv_mov_product_date (product_id, created_at),
62     INDEX idx_inv_mov_type (type)
63 );
64
65 CREATE TABLE IF NOT EXISTS sales_daily (
66     id BIGINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
67     sale_date DATE NOT NULL,
68     product_id BIGINT UNSIGNED NOT NULL,
69     qty DECIMAL(12,3) NOT NULL,
70     unit ENUM('kg','g','piece','pack','box','lt','ml') NOT NULL DEFAULT 'piece',
71     category_off VARCHAR(120) NULL,
72     perecedero BOOLEAN NOT NULL DEFAULT 0,
73     en_temporada BOOLEAN NOT NULL DEFAULT 0,
74     temp_inicio_mes TINYINT NULL,
75     temp_fin_mes TINYINT NULL,
76     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
77
78     CONSTRAINT fk_sales_daily_product
79     FOREIGN KEY (product_id) REFERENCES products(id)
80     ON DELETE CASCADE,

```

```

79     UNIQUE KEY uq_sales_daily (sale_date, product_id, unit),
80     INDEX idx_sales_daily_date (sale_date),
81     INDEX idx_sales_daily_product_date (product_id, sale_date)
82 );

```

A.8. Código fuente del backend

A.8.1. Módulo models

Archivo: sale.py

```

1  from sqlalchemy import Column, BigInteger, DateTime, DECIMAL, String
2  from sqlalchemy.orm import relationship
3  from sqlalchemy.sql import func
4  from app.db import Base
5
6  class Sale(Base):
7      __tablename__ = "sales"
8
9      id = Column(BigInteger, primary_key=True, autoincrement=True)
10     sold_at = Column(DateTime, nullable=False,
11                      ↪ server_default=func.current_timestamp())
12     total = Column(DECIMAL(12, 2), nullable=False, server_default="0")
13     payment_method = Column(String(40), nullable=True)
14     notes = Column(String(255), nullable=True)
15
16     items = relationship("SaleItem", back_populates="sale", cascade="all,
17                      ↪ delete-orphan")

```

Archivo: sale_item.py

```

1  from sqlalchemy import Column, BigInteger, ForeignKey, DECIMAL, Enum
2  from sqlalchemy.orm import relationship
3  from app.db import Base
4
5  class SaleItem(Base):
6      __tablename__ = "sale_items"
7
8      id = Column(BigInteger, primary_key=True, autoincrement=True)
9      sale_id = Column(BigInteger, ForeignKey("sales.id", ondelete="CASCADE"),
10                     ↪ nullable=False)
11     product_id = Column(BigInteger, ForeignKey("products.id"), nullable=False)
12
13     qty = Column(DECIMAL(12, 3), nullable=False)

```

```

13     unit = Column(Enum("kg", "g", "piece", "pack", "box", "lt", "ml",
    ↪     name="sale_items_unit"), nullable=False, server_default="piece")
14     unit_price = Column(DECIMAL(12, 2), nullable=False)
15     subtotal = Column(DECIMAL(12, 2), nullable=False)
16
17     sale = relationship("Sale", back_populates="items")

```

Archivo: product.py

```

1     from sqlalchemy import Column, Integer, String, Boolean, DateTime, ForeignKey,
    ↪     DECIMAL, Enum, func
2     from sqlalchemy.orm import relationship
3     from app.db import Base
4
5     class Product(Base):
6         __tablename__ = "products"
7
8         id = Column(Integer, primary_key=True, autoincrement=True)
9
10        # Datos base del producto
11        product_name = Column(String(255), nullable=False)
12        category_off = Column(String(120), nullable=True)
13        shelf_life_pantry_days = Column(Integer, nullable=True)
14        shelf_life_fridge_days = Column(Integer, nullable=True)
15        shelf_life_freezer_days = Column(Integer, nullable=True)
16        perecedero = Column(Boolean, nullable=False, default=False)
17        created_at = Column(DateTime, nullable=False, server_default=func.now())

```

Archivo: inventory_movement.py

```

1     from sqlalchemy import Column, BigInteger, ForeignKey, DECIMAL, Enum, String,
    ↪     TIMESTAMP
2     from sqlalchemy.sql import func
3     from app.db import Base
4
5     class InventoryMovement(Base):
6         __tablename__ = "inventory_movements"
7
8         id = Column(BigInteger, primary_key=True, autoincrement=True)
9         product_id = Column(BigInteger, ForeignKey("products.id", ondelete="CASCADE"),
    ↪         nullable=False)
10        type = Column(Enum("IN", "OUT", "ADJUST", name="inv_mov_type"), nullable=False)
11        qty = Column(DECIMAL(12, 3), nullable=False)
12        unit = Column(Enum("kg", "g", "piece", "pack", "box", "lt", "ml",
    ↪         name="inv_mov_unit"), nullable=False, server_default="piece")

```



```
13     unit_cost = Column(DECIMAL(12, 2), nullable=True)
14     reason = Column(String(255), nullable=True)
15     created_at = Column(TIMESTAMP, nullable=False,
16         ↳ server_default=func.current_timestamp())
```

A.8.2. Modulo schemas

Archivo: stats.py

```
1     from pydantic import BaseModel
2
3     class StatPoint(BaseModel):
4         date: str
5         value: float
6
7     class ProductStatsOut(BaseModel):
8         product_id: int
9         product_name: str
10        unit: str
11        range: str
12        metric: str
13
14        total: float
15        avg_per_day: float
16        volatility: float
17
18        series: list[StatPoint]
```

Archivo: sales_history.py

```
1     from datetime import datetime
2     from pydantic import BaseModel
3
4
5     class SaleHistoryRow(BaseModel):
6         sale_id: int
7         item_id: int
8
9         sold_at: datetime
10        product_id: int
11        product_name: str
12
13        qty: float
```

```
14     unit: str
15     unit_price: float
16     subtotal: float
17
18     class Config:
19         from_attributes = True
20
```

Archivo: sale.py

```
1     from pydantic import BaseModel, Field
2     from typing import List, Optional
3     from datetime import datetime
4
5
6     class SaleItemCreate(BaseModel):
7         product_id: int
8         qty: float = Field(gt=0)
9         unit: str = "piece"
10        unit_price: float = Field(ge=0)
11
12    class SaleCreate(BaseModel):
13        sold_at: Optional[datetime] = None
14        payment_method: Optional[str] = None
15        notes: Optional[str] = None
16        items: List[SaleItemCreate] = Field(min_length=1)
17
18
19    class SaleItemOut(BaseModel):
20        id: int
21        product_id: int
22        qty: float
23        unit: str
24        unit_price: float
25        subtotal: float
26
27    class Config:
28        from_attributes = True
29
30    class SaleOut(BaseModel):
31        id: int
32        sold_at: datetime
33        total: float
34        payment_method: Optional[str] = None
35        notes: Optional[str] = None
36        items: List[SaleItemOut]
37
```

```
38     class Config:
39         from_attributes = True
```

Archivo: product.py

```
1     from pydantic import BaseModel
2
3     class ProductCreate(BaseModel):
4         product_name: str
5         category_off: str
6         shelf_life_pantry_days: int = 0
7         shelf_life_fridge_days: int = 0
8         shelf_life_freezer_days: int = 0
9
10    class ProductOut(BaseModel):
11        id: int
12        perecedero: int
13
14    class Config:
15        from_attributes = True
16
17    class ProductUpdate(BaseModel):
18        product_name: str
19        category_off: str
20        shelf_life_pantry_days: int = 0
21        shelf_life_fridge_days: int = 0
22        shelf_life_freezer_days: int = 0
23
```

Archivo: ml.py

```
1     from pydantic import BaseModel, Field
2     from typing import List, Optional
3
4     class PredictRequest(BaseModel):
5         product_id: int = Field(..., ge=1)
6         unit: str = Field("piece") # piece/kg/g/pack/box/lt/ml
7         days: int = Field(7, ge=1, le=60)
8         price: Optional[float] = Field(None, ge=0)
9
10    class PredPoint(BaseModel):
11        date: str
12        y_pred: float
13
14    class PredictResponse(BaseModel):
```

```
15     product_id: int
16     product_name: str
17     unit: str
18     days: int
19     used_price: float
20     points: List[PredPoint]
21
```

Archivo: inventory.py

```
1     from pydantic import BaseModel, Field
2     from typing import Literal, Optional
3     from datetime import datetime
4
5     MovementType = Literal["IN", "OUT", "ADJUST"]
6     UnitType = Literal["kg", "g", "piece", "pack", "box", "lt", "ml"]
7
8     class InventoryMovementCreate(BaseModel):
9         product_id: int
10        type: MovementType
11        qty: float = Field(gt=0)
12        unit: UnitType = "piece"
13        unit_cost: Optional[float] = None
14        reason: Optional[str] = None
15
16    class InventoryMovementOut(BaseModel):
17        id: int
18        product_id: int
19        type: MovementType
20        qty: float
21        unit: UnitType
22        unit_cost: Optional[float] = None
23        reason: Optional[str] = None
24        created_at: datetime
25
26    class Config:
27        from_attributes = True
28
29    class StockRowOut(BaseModel):
30        product_id: int
31        product_name: str
32        category_off: Optional[str] = None
33        perecedero: int
34        unit: UnitType
35        stock_actual: float
36
```

A.8.3. Modulo routers

Archivo: stats.py

```
1  from fastapi import APIRouter, Depends, HTTPException
2  from sqlalchemy.orm import Session
3
4  from app.db import get_db
5  from app.schemas.stats import ProductStatsOut
6  from app.services.stats_service import get_product_stats
7
8  router = APIRouter(prefix="/stats", tags=["Stats"])
9
10 @router.get("/product", response_model=ProductStatsOut)
11 def stats_product(
12     product_id: int | None = None,
13     product_name: str | None = None,
14     unit: str = "piece",
15     range: str = "30d",
16     metric: str = "sales",
17     db: Session = Depends(get_db),
18 ):
19     try:
20         return get_product_stats(db, product_id, product_name, unit, range,
21                                 ↪ metric)
22     except ValueError as e:
23         raise HTTPException(status_code=404, detail=str(e))
```

Archivo: sales.py

```
1  from fastapi import APIRouter, Depends
2  from sqlalchemy.orm import Session
3
4  from app.db import get_db
5  from app.schemas.sale import SaleCreate, SaleOut
6  from app.services.sales_service import create_sale
7
8  router = APIRouter(prefix="/sales", tags=["sales"])
9
10 @router.post("", response_model=SaleOut)
11 def post_sale(payload: SaleCreate, db: Session = Depends(get_db)):
12     return create_sale(db, payload.model_dump())
13
```

Archivo: sales_history.py

```

1  from datetime import date
2  from fastapi import APIRouter, Depends, Query
3  from sqlalchemy.orm import Session
4
5  from app.db import get_db
6  from app.schemas.sales_history import SaleHistoryRow
7  from app.services.sales_history_service import get_sales_history
8
9  router = APIRouter(prefix="/sales", tags=["sales"])
10
11
12  @router.get("/history", response_model=list[SaleHistoryRow])
13  def sales_history(
14      db: Session = Depends(get_db),
15      from_date: date | None = Query(default=None, alias="from"),
16      to_date: date | None = Query(default=None, alias="to"),
17      q: str | None = Query(default=None),
18      limit: int = Query(default=200, ge=1, le=2000),
19      offset: int = Query(default=0, ge=0),
20  ):
21      return get_sales_history(
22          db=db,
23          from_date=from_date,
24          to_date=to_date,
25          q=q,
26          limit=limit,
27          offset=offset,
28      )

```

Archivo: products.py

```

1  from fastapi import APIRouter, Depends, Response
2  from sqlalchemy.orm import Session
3  from app.db import get_db
4  from app.schemas.product import ProductCreate, ProductOut, ProductUpdate
5  from app.services.products_service import crear_producto, listar_productos,
6      ↪ actualizar_producto, eliminar_producto
7
8
9  router = APIRouter(prefix="/products", tags=["products"])
10
11  @router.post("", response_model=ProductOut)
12  def post_product(payload: ProductCreate, db: Session = Depends(get_db)):
13      return crear_producto(db, payload.model_dump())
14
15
16  @router.get("", response_model=list[ProductOut])

```

```

15     def get_products(db: Session = Depends(get_db)):
16         return listar_productos(db)
17
18
19     @router.put("/{product_id}", response_model=ProductOut)
20     def put_product(product_id: int, payload: ProductUpdate, db: Session =
    ↳ Depends(get_db)):
21         return actualizar_producto(db, product_id, payload.model_dump())
22
23     @router.delete("/{product_id}", status_code=204)
24     def delete_product(product_id: int, db: Session = Depends(get_db)):
25         eliminar_producto(db, product_id)
26         return Response(status_code=204)

```

Archivo: ml.py

```

1     from fastapi import APIRouter, Depends, HTTPException
2     from sqlalchemy.orm import Session
3
4     from app.db import get_db
5     from app.schemas.ml import PredictRequest, PredictResponse
6     from app.services.ml_service import predict_sales
7
8     router = APIRouter(prefix="/ml", tags=["ml"])
9
10    @router.post("/predict", response_model=PredictResponse)
11    def predict(req: PredictRequest, db: Session = Depends(get_db)) ->
    ↳ PredictResponse:
12        try:
13            return predict_sales(db, req)
14        except ValueError as e:
15            raise HTTPException(status_code=400, detail=str(e))
16        except Exception as e:
17            raise HTTPException(status_code=500, detail=f"Error interno: {e}")
18

```

Archivo: inventory.py

```

1     from fastapi import APIRouter, Depends
2     from sqlalchemy.orm import Session
3
4     from app.db import get_db
5     from app.schemas.inventory import InventoryMovementCreate, InventoryMovementOut,
    ↳ StockRowOut
6     from app.services.inventory_service import crear_movimiento, obtener_stock

```

```

7
8     router = APIRouter(prefix="/inventory", tags=["Inventory"])
9
10    @router.post("/movements", response_model=InventoryMovementOut)
11    def post_movement(payload: InventoryMovementCreate, db: Session =
12        ↳ Depends(get_db)):
13        return crear_movimiento(db, payload.model_dump())
14
15    @router.get("/stock", response_model=list[StockRowOut])
16    def get_stock(db: Session = Depends(get_db)):
17        return obtener_stock(db)

```

Archivo: dashboard.py

```

1     from fastapi import APIRouter, Depends, Query
2     from sqlalchemy.orm import Session
3
4     from app.db import get_db
5     from app.services.dashboard_service import (
6         get_dashboard_summary,
7         get_dashboard_trend,
8     )
9
10    router = APIRouter(
11        prefix="/dashboard",
12        tags=["Dashboard"]
13    )
14
15
16    @router.get("/summary")
17    def dashboard_summary(db: Session = Depends(get_db)):
18        return get_dashboard_summary(db)
19
20
21    @router.get("/trend")
22    def dashboard_trend(
23        days: int = Query(7, ge=1, le=60),
24        db: Session = Depends(get_db),
25    ):
26        return get_dashboard_trend(db, days)
27

```

A.8.4. Modulo services

Archivo: stats_service.py


```
1  from sqlalchemy.orm import Session
2  from sqlalchemy import text
3
4  RANGE_TO_DAYS = {"7d": 7, "30d": 30, "90d": 90, "365d": 365}
5
6  def _resolve_product_id(db: Session, product_id: int | None, product_name: str |
7  ↪ None) -> tuple[int, str]:
8      if product_id is not None:
9          row = db.execute(
10             text("SELECT id, product_name FROM products WHERE id=:id LIMIT 1"),
11             {"id": product_id},
12         ).mappings().first()
13         if not row:
14             raise ValueError("Producto no encontrado por id")
15         return int(row["id"]), row["product_name"]
16
17     if product_name:
18         row = db.execute(
19             text("SELECT id, product_name FROM products WHERE product_name=:name
20             ↪ LIMIT 1"),
21             {"name": product_name},
22         ).mappings().first()
23         if not row:
24             raise ValueError("Producto no encontrado por nombre")
25         return int(row["id"]), row["product_name"]
26
27     raise ValueError("Debes enviar product_id o product_name")
28
29 def get_product_stats(
30     db: Session,
31     product_id: int | None,
32     product_name: str | None,
33     unit: str = "piece",
34     range_: str = "30d",
35     metric: str = "sales",
36 ):
37     days = RANGE_TO_DAYS.get(range_, 30)
38
39     pid, pname = _resolve_product_id(db, product_id, product_name)
40
41     # value por día (sales = qty, revenue = qty * price)
42     if metric == "revenue":
43         value_expr = "SUM(qty * COALESCE(unit_price_avg, 0))"
44     else:
45         value_expr = "SUM(qty)"
46
47     rows = db.execute(
```

```

47         text(f"""
48             SELECT
49                 sale_date,
50                 {value_expr} AS value
51             FROM v_sales_daily_unified
52             WHERE product_id = :pid
53                 AND unit = :unit
54                 AND sale_date >= (CURRENT_DATE - INTERVAL :days DAY)
55             GROUP BY sale_date
56             ORDER BY sale_date ASC
57         """),
58         {"pid": pid, "unit": unit, "days": days},
59     ).mappings().all()
60
61     series = [{"date": str(r["sale_date"]), "value": float(r["value"] or 0.0)} for
62               ↪ r in rows]
63
64     # KPIs
65     values = [p["value"] for p in series]
66     total = float(sum(values))
67     n = len(values)
68     avg = float(total / n) if n else 0.0
69
70     if n:
71         mean = avg
72         var = sum((x - mean) ** 2 for x in values) / n
73         volatility = float(var ** 0.5)
74     else:
75         volatility = 0.0
76
77     return {
78         "product_id": pid,
79         "product_name": pname,
80         "unit": unit,
81         "range": range_,
82         "metric": metric,
83         "total": total,
84         "avg_per_day": avg,
85         "volatility": volatility,
86         "series": series,
87     }

```

Archivo: sales_service.py

```

1     from sqlalchemy.orm import Session, joinedload
2     from fastapi import HTTPException

```

```
3 from datetime import datetime
4 from zoneinfo import ZoneInfo
5
6 from app.models.sale import Sale
7 from app.models.sale_item import SaleItem
8 from app.models.inventory_movement import InventoryMovement
9
10 def create_sale(db: Session, payload: dict) -> Sale:
11     items = payload.get("items") or []
12     if not items:
13         raise HTTPException(status_code=400, detail="La venta debe tener al menos
14             ↳ 1 item.")
15
16     sale = Sale(
17         sold_at=payload.get("sold_at") or
18             ↳ datetime.now(ZoneInfo("America/Mexico_City")),
19         payment_method=payload.get("payment_method"),
20         notes=payload.get("notes"),
21         total=0,
22     )
23
24     try:
25         db.add(sale)
26         db.flush() # para obtener sale.id
27
28         total = 0.0
29
30         for it in items:
31             qty = float(it["qty"])
32             unit_price = float(it["unit_price"])
33             subtotal = round(qty * unit_price, 2)
34             total += subtotal
35
36             si = SaleItem(
37                 sale_id=sale.id,
38                 product_id=int(it["product_id"]),
39                 qty=qty,
40                 unit=it.get("unit", "piece"),
41                 unit_price=unit_price,
42                 subtotal=subtotal,
43             )
44             db.add(si)
45
46             # registrar movimiento OUT
47             mov = InventoryMovement(
48                 product_id=int(it["product_id"]),
49                 type="OUT",
50                 qty=qty,
51                 unit=it.get("unit", "piece"),
```

```

50         unit_cost=None,
51         reason=f"SALE #{sale.id}",
52     )
53     db.add(mov)
54
55     sale.total = round(total, 2)
56
57     db.commit()
58
59     except Exception as e:
60         db.rollback()
61         raise
62
63     # devolver con items cargados
64     sale_db = (
65         db.query(Sale)
66         .options(joinedload(Sale.items))
67         .filter(Sale.id == sale.id)
68         .first()
69     )
70     return sale_db
71

```

Archivo: sales_history_service.py

```

1  from datetime import date, datetime, timedelta
2  from sqlalchemy import select, or_
3  from sqlalchemy.orm import Session
4
5  from app.models.sale import Sale
6  from app.models.sale_item import SaleItem
7  from app.models.product import Product
8
9
10 def _parse_range(from_date: date | None, to_date: date | None):
11     """
12     Convertimos from/to (DATE) a rango DATETIME:
13     - desde: 00:00:00 del día
14     - hasta: EXCLUSIVO (día siguiente 00:00:00) para que 'to' sea inclusivo
15     """
16     dt_from = datetime.min
17     dt_to_excl = datetime.max
18
19     if from_date:
20         dt_from = datetime.combine(from_date, datetime.min.time())
21
22     if to_date:

```

```

23         dt_to_excl = datetime.combine(to_date + timedelta(days=1),
24                                     ↪ datetime.min.time())
25
26     return dt_from, dt_to_excl
27
28     def get_sales_history(
29         db: Session,
30         from_date: date | None = None,
31         to_date: date | None = None,
32         q: str | None = None,
33         limit: int = 200,
34         offset: int = 0,
35     ):
36         dt_from, dt_to_excl = _parse_range(from_date, to_date)
37
38         stmt = (
39             select(
40                 Sale.id.label("sale_id"),
41                 SaleItem.id.label("item_id"),
42                 Sale.sold_at.label("sold_at"),
43                 Product.id.label("product_id"),
44                 Product.product_name.label("product_name"),
45                 SaleItem.qty.label("qty"),
46                 SaleItem.unit.label("unit"),
47                 SaleItem.unit_price.label("unit_price"),
48                 SaleItem.subtotal.label("subtotal"),
49             )
50             .join(SaleItem, SaleItem.sale_id == Sale.id)
51             .join(Product, Product.id == SaleItem.product_id)
52             .where(Sale.sold_at >= dt_from)
53             .where(Sale.sold_at < dt_to_excl)
54             .order_by(Sale.sold_at.desc(), SaleItem.id.desc())
55             .limit(limit)
56             .offset(offset)
57         )
58
59         if q:
60             term = f"%{q.strip()}%"
61             stmt = stmt.where(
62                 or_(
63                     Product.product_name.ilike(term),
64                     Product.category_off.ilike(term),
65                 )
66             )
67
68         rows = db.execute(stmt).mappings().all()
69         return rows

```

70

Archivo: products_service.py

```
1  from sqlalchemy.orm import Session
2  from app.models.product import Product
3  from app.ml.modelo_perecedero import predecir_perecedero
4  from fastapi import HTTPException
5
6  def crear_producto(db: Session, payload: dict) -> Product:
7      """Crea un producto en BD calculando perecedero con el modelo ML."""
8      perecedero = predecir_perecedero(payload)
9
10     nuevo = Product(
11         product_name=payload["product_name"],
12         category_off=payload["category_off"],
13         shelf_life_pantry_days=payload.get("shelf_life_pantry_days", 0),
14         shelf_life_fridge_days=payload.get("shelf_life_fridge_days", 0),
15         shelf_life_freezer_days=payload.get("shelf_life_freezer_days", 0),
16         perecedero=perecedero,
17     )
18
19     db.add(nuevo)
20     db.commit()
21     db.refresh(nuevo)
22     return nuevo
23
24
25 def listar_productos(db: Session) -> list[Product]:
26     return db.query(Product)
27
28
29 def actualizar_producto(db: Session, product_id: int, payload: dict) -> Product:
30     producto = db.query(Product).filter(Product.id == product_id).first()
31     if not producto:
32         raise HTTPException(status_code=404, detail="Producto no encontrado")
33
34     # Recalcular perecedero con ML
35     perecedero = predecir_perecedero(payload)
36
37     producto.product_name = payload["product_name"]
38     producto.category_off = payload["category_off"]
39     producto.shelf_life_pantry_days = payload.get("shelf_life_pantry_days", 0)
40     producto.shelf_life_fridge_days = payload.get("shelf_life_fridge_days", 0)
41     producto.shelf_life_freezer_days = payload.get("shelf_life_freezer_days", 0)
42     producto.perecedero = perecedero
43
```

```
44         db.commit()
45         db.refresh(producto)
46         return producto
47
48     def eliminar_producto(db: Session, product_id: int) -> Product:
49         producto = db.query(Product).filter(Product.id == product_id).first()
50         if not producto:
51             raise HTTPException(status_code=404, detail="Producto no encontrado")
52
53         db.delete(producto)
54         db.commit()
55
```

Archivo: ml_service.py

```
1     from datetime import date, timedelta
2     from typing import Dict, List, Tuple, Set
3     import numpy as np
4     from sqlalchemy.orm import Session
5     from sqlalchemy import text
6     from app.ml.predictor import load_artifacts, predict_one
7     from app.schemas.ml import PredictRequest, PredictResponse, PredPoint
8
9     UNITS_ALLOWED = {"piece", "kg", "g", "pack", "box", "lt", "ml"}
10
11     _ARTIFACTS = None
12
13     def _get_artifacts():
14         global _ARTIFACTS
15         if _ARTIFACTS is None:
16             model, scaler, meta = load_artifacts()
17             _ARTIFACTS = (model, scaler, meta)
18         return _ARTIFACTS
19
20     def _ymd(d: date) -> str:
21         return d.isoformat()
22
23     def _safe_mean(xs: List[float]) -> float:
24         return float(sum(xs) / len(xs)) if xs else 0.0
25
26     def _rolling_mean(series: List[float], n: int) -> float:
27         if n <= 0:
28             return 0.0
29         return _safe_mean(series[-n:])
30
31     def _factorize_lookup(values_list: List[str], key: str) -> int:
32         try:
```

```

33         return values_list.index(key)
34     except ValueError:
35         return -1
36
37 def _fetch_product(db: Session, product_id: int) -> Dict:
38     row = db.execute(
39         text("""
40             SELECT id, product_name, category_off, perecedero
41             FROM products
42             WHERE id = :pid
43             """),
44         {"pid": product_id}
45     ).mappings().first()
46
47     if not row:
48         raise ValueError(f"Producto no existe: product_id={product_id}")
49     return dict(row)
50
51 def _fetch_last_price(db: Session, product_id: int, unit: str) -> float:
52     row = db.execute(
53         text("""
54             SELECT unit_price_avg
55             FROM v_sales_daily_unified
56             WHERE product_id = :pid AND unit = :unit AND unit_price_avg IS NOT
57               ↪ NULL
58             ORDER BY sale_date DESC
59             LIMIT 1
60             """),
61         {"pid": product_id, "unit": unit}
62     ).mappings().first()
63
64     return float(row["unit_price_avg"]) if row and row["unit_price_avg"] is not
65     ↪ None else 0.0
66
67 def _fetch_event_dates(db: Session, product_id: int, start: date, end: date) ->
68     ↪ Set[date]:
69     rows = db.execute(
70         text("""
71             SELECT e.date AS d
72             FROM events e
73             JOIN event_products ep ON ep.event_id = e.id
74             WHERE ep.product_id = :pid
75             AND e.date BETWEEN :d1 AND :d2
76             """),
77         {"pid": product_id, "d1": start, "d2": end}
78     ).mappings().all()
79
80     # d normalmente ya es date; si fuera datetime, normalize:
81     out = set()

```



```

79     for r in rows:
80         d = r["d"]
81         out.add(d if isinstance(d, date) else d.date())
82     return out
83
84 def _fetch_history(db: Session, product_id: int, unit: str, days_back: int = 140)
↪ -> List[Tuple[date, float]]:
85     rows = db.execute(
86         text("""
87             SELECT sale_date, SUM(qty) AS qty
88             FROM v_sales_daily_unified
89             WHERE product_id = :pid
90                 AND unit = :unit
91                 AND sale_date >= (CURRENT_DATE - INTERVAL :days DAY)
92             GROUP BY sale_date
93             ORDER BY sale_date ASC
94         """),
95         {"pid": product_id, "unit": unit, "days": days_back}
96     ).mappings().all()
97
98     out = []
99     for r in rows:
100         sd = r["sale_date"]
101         sd = sd if isinstance(sd, date) else sd.date()
102         out.append((sd, float(r["qty"] or 0)))
103     return out
104
105 def _dense_daily_series(hist: List[Tuple[date, float]], end_date: date,
↪ days_needed: int = 100) -> List[float]:
106     if not hist:
107         return []
108     start_date = end_date - timedelta(days=days_needed - 1)
109     dmap = {d: v for d, v in hist}
110
111     series = []
112     d = start_date
113     while d <= end_date:
114         series.append(float(dmap.get(d, 0.0)))
115         d += timedelta(days=1)
116     return series
117
118 def predict_sales(db: Session, req: PredictRequest) -> PredictResponse:
119     if req.unit not in UNITS_ALLOWED:
120         raise ValueError(f"Unidad inválida: {req.unit}")
121
122     model, scaler, meta = _get_artifacts()
123     features = list(meta["features"])
124
125     prod = _fetch_product(db, req.product_id)

```

```

126     product_name = prod.get("product_name") or ""
127     category_off = prod.get("category_off") or ""
128     perecedero = int(bool(prod.get("perecedero")))
129
130     product_uniques = list(meta["product_uniques"])
131     category_uniques = list(meta["category_uniques"])
132
133     pid_feat = _factorize_lookup(product_uniques, product_name)
134     cid_feat = _factorize_lookup(category_uniques, category_off)
135
136     if pid_feat < 0:
137         raise ValueError(
138             f"Producto no existe en el mapeo del modelo (meta).
139             ↪ product_name='{product_name}'."
140         )
141
142     used_price = float(req.price) if req.price is not None else
143     ↪ _fetch_last_price(db, req.product_id, req.unit)
144
145     hist = _fetch_history(db, req.product_id, req.unit, days_back=160)
146     today = date.today()
147
148     base_series = _dense_daily_series(hist, end_date=today, days_needed=120)
149     if len(base_series) < 90:
150         raise ValueError("No hay suficiente histórico para predecir (~90 días).")
151
152     ev_dates = _fetch_event_dates(db, req.product_id, today + timedelta(days=1),
153     ↪ today + timedelta(days=req.days))
154
155     points: List[PredPoint] = []
156     series = base_series[:]
157
158     for i in range(req.days):
159         d = today + timedelta(days=i + 1)
160
161         anio = d.year
162         mes = d.month
163         dia_semana = d.weekday()
164         es_fin_semana = 1 if dia_semana in (5, 6) else 0
165         mes_sin = float(np.sin(2 * np.pi * mes / 12))
166         mes_cos = float(np.cos(2 * np.pi * mes / 12))
167         dow_sin = float(np.sin(2 * np.pi * dia_semana / 7))
168         dow_cos = float(np.cos(2 * np.pi * dia_semana / 7))
169
170         en_temporada = 0
171         hay_evento = 1 if d in ev_dates else 0
172
173         lag_1 = series[-1]
174         lag_7 = series[-7] if len(series) >= 7 else 0.0

```

```

172     lag_14 = series[-14] if len(series) >= 14 else 0.0
173     media_7 = _rolling_mean(series[:-1], 7)
174     media_28 = _rolling_mean(series[:-1], 28)
175     media_90 = _rolling_mean(series[:-1], 90)
176
177     row_map = {
178         "product_id": float(pid_feat),
179         "category_id": float(max(cid_feat, 0)),
180         "perecedero": float(perecedero),
181         "precio": float(used_price),
182         "en_temporada": float(en_temporada),
183         "hay_evento": float(hay_evento),
184         "anio": float(anio),
185         "mes": float(mes),
186         "dia_semana": float(dia_semana),
187         "es_fin_semana": float(es_fin_semana),
188         "mes_sin": float(mes_sin),
189         "mes_cos": float(mes_cos),
190         "dow_sin": float(dow_sin),
191         "dow_cos": float(dow_cos),
192         "lag_1": float(lag_1),
193         "lag_7": float(lag_7),
194         "lag_14": float(lag_14),
195         "media_7": float(media_7),
196         "media_28": float(media_28),
197         "media_90": float(media_90),
198     }
199
200     x = np.array([row_map[f] for f in features], dtype="float32")
201
202     y_pred = float(predict_one(x))
203
204     points.append(PredPoint(date=_ymd(d), y_pred=y_pred))
205     series.append(y_pred)
206
207     return PredictResponse(
208         product_id=req.product_id,
209         product_name=product_name,
210         unit=req.unit,
211         days=req.days,
212         used_price=float(used_price),
213         points=points,
214     )
215

```

Archivo: inventory_service.py

```

1     from sqlalchemy.orm import Session

```

```
2 from sqlalchemy import func, case
3 from fastapi import HTTPException
4
5 from app.models.product import Product
6 from app.models.inventory_movement import InventoryMovement
7
8 def crear_movimiento(db: Session, payload: dict) -> InventoryMovement:
9     prod = db.query(Product).filter(Product.id == payload["product_id"]).first()
10     if not prod:
11         raise HTTPException(status_code=404, detail="Producto no encontrado")
12
13     mov = InventoryMovement(
14         product_id=payload["product_id"],
15         type=payload["type"],
16         qty=payload["qty"],
17         unit=payload.get("unit", "piece"),
18         unit_cost=payload.get("unit_cost"),
19         reason=payload.get("reason"),
20     )
21
22     db.add(mov)
23     db.commit()
24     db.refresh(mov)
25     return mov
26
27 def obtener_stock(db: Session):
28     """
29     Stock por product.
30     stock = IN + ADJUST - OUT
31     """
32
33     signed_qty = case(
34         (InventoryMovement.type.in_(["IN", "ADJUST"]), InventoryMovement.qty),
35         (InventoryMovement.type == "OUT", -InventoryMovement.qty),
36         else_=0
37     )
38
39     rows = (
40         db.query(
41             Product.id.label("product_id"),
42             Product.product_name,
43             Product.category_off,
44             Product.perecedero,
45             InventoryMovement.unit.label("unit"),
46             func.coalesce(func.sum(signed_qty), 0).label("stock_actual"),
47         )
48         .outerjoin(InventoryMovement, InventoryMovement.product_id == Product.id)
49         .group_by(
50             Product.id,
```

```

51         Product.product_name,
52         Product.category_off,
53         Product.perecedero,
54         InventoryMovement.unit,
55     )
56     .order_by(Product.product_name.asc())
57     .all()
58 )
59
60 result = []
61 for r in rows:
62     if r.unit is None:
63         # producto sin movimientos aún
64         result.append({
65             "product_id": int(r.product_id),
66             "product_name": r.product_name,
67             "category_off": r.category_off,
68             "perecedero": int(r.perecedero),
69             "unit": "piece",
70             "stock_actual": 0.0,
71         })
72     else:
73         result.append({
74             "product_id": int(r.product_id),
75             "product_name": r.product_name,
76             "category_off": r.category_off,
77             "perecedero": int(r.perecedero),
78             "unit": r.unit,
79             "stock_actual": float(r.stock_actual or 0),
80         })
81
82 return result
83

```

Archivo: dashboard_service.py

```

1  from datetime import date
2  from sqlalchemy.orm import Session
3  from sqlalchemy import text
4
5
6  def get_dashboard_summary(db: Session) -> dict:
7      """
8      KPIs del día actual + alertas rápidas
9      (sin merma estimada)
10     """
11     today = date.today()

```

```

12
13     # Ventas del día (usa sales.total)
14     row = db.execute(
15         text("""
16             SELECT
17                 COALESCE(SUM(s.total), 0) AS sales_mxn,
18                 COUNT(DISTINCT s.id) AS tickets,
19                 COALESCE(SUM(si.qty), 0) AS items_sold
20             FROM sales s
21             LEFT JOIN sale_items si ON si.sale_id = s.id
22             WHERE DATE(s.sold_at) = :today
23         """),
24         {"today": today}
25     ).mappings().first() or {
26         "sales_mxn": 0,
27         "tickets": 0,
28         "items_sold": 0,
29     }
30
31     # Alertas: stock crítico (por producto, no por unidad)
32     critical_stock = db.execute(
33         text("""
34             SELECT COUNT(*) AS n
35             FROM (
36                 SELECT v.product_id
37                 FROM v_stock_actual v
38                 GROUP BY v.product_id
39                 HAVING SUM(v.stock_actual) <= 0
40             ) t
41         """)
42     ).scalar() or 0
43
44     # Reorden sugerido (stock bajo por producto)
45     reorder_suggested = db.execute(
46         text("""
47             SELECT COUNT(*) AS n
48             FROM (
49                 SELECT v.product_id
50                 FROM v_stock_actual v
51                 GROUP BY v.product_id
52                 HAVING SUM(v.stock_actual) > 0
53                     AND SUM(v.stock_actual) <= 5
54             ) t
55         """)
56     ).scalar() or 0
57
58     # Productos en temporada hoy (si sales_daily tiene datos)
59     in_season = db.execute(
60         text("""

```

```

61         SELECT COUNT(DISTINCT sd.product_id) AS n
62         FROM sales_daily sd
63         WHERE sd.sale_date = :today
64         AND sd.en_temporada = 1
65         """),
66         {"today": today}
67     ).scalar() or 0
68
69     return {
70         "date": today.isoformat(),
71         "sales_mxn": float(row["sales_mxn"] or 0),
72         "tickets": int(row["tickets"] or 0),
73         "items_sold": float(row["items_sold"] or 0),
74         "alerts": {
75             "critical_stock": int(critical_stock),
76             "reorder_suggested": int(reorder_suggested),
77             "in_season": int(in_season),
78         },
79     }
80
81
82 def get_dashboard_trend(db: Session, days: int = 7) -> list:
83     """
84     Serie diaria de ventas (MXN)
85     """
86     rows = db.execute(
87         text("""
88             SELECT
89                 DATE(s.sold_at) AS d,
90                 COALESCE(SUM(s.total), 0) AS value
91             FROM sales s
92             WHERE s.sold_at >= (CURRENT_DATE - INTERVAL :days DAY)
93             GROUP BY DATE(s.sold_at)
94             ORDER BY d ASC
95         """),
96         {"days": days}
97     ).mappings().all()
98
99     return [
100         {
101             "date": r["d"].isoformat(),
102             "value": float(r["value"] or 0)
103         }
104         for r in rows
105     ]
106

```

A.9. Integración del modelo predictivo con el backend

```
1  from pathlib import Path
2  import joblib
3  import numpy as np
4  import tensorflow as tf
5
6  MODELS_DIR = Path(__file__).resolve().parents[1] / "ml_models"
7  MODEL_PATH = MODELS_DIR / "nn_sales_model.keras"
8  SCALER_PATH = MODELS_DIR / "scaler.joblib"
9  META_PATH = MODELS_DIR / "meta.joblib"
10
11  _model = None
12  _scaler = None
13  _meta = None
14
15  def load_artifacts():
16      global _model, _scaler, _meta
17      if _model is None:
18          _model = tf.keras.models.load_model(MODEL_PATH)
19      if _scaler is None:
20          _scaler = joblib.load(SCALER_PATH)
21      if _meta is None:
22          _meta = joblib.load(META_PATH)
23      return _model, _scaler, _meta
24
25  def predict_one(x_row: np.ndarray) -> float:
26      model, scaler, meta = load_artifacts()
27      X = x_row.astype("float32", copy=False).reshape(1, -1)
28
29      num_idx = meta["num_idx"]
30      X[:, num_idx] = scaler.transform(X[:, num_idx])
31
32      y_pred_log = model.predict(X, verbose=0).ravel()[0]
33      y_pred = float(np.expml(y_pred_log))
34      if y_pred < 0:
35          y_pred = 0.0
36      return y_pred
37
```
